claude-windows / c7679509-ee2b-4443-b240-0b3b1b1a7291

Metadata

- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\c7679509-ee2b-4443-b240-0b3b1b1a7291\subagents\workflows\wf_0f586cf2-9eb\agent-aad4e1b75910ad6bf.jsonl`
- SHA-256: `fabee1ccd59ea7f06efa994ef35f3a0f754d7091f5fe50c21a1c6335ff21ba56`
- Source modified: `2026-06-21T14:45:46+00:00`
- Imported at: `2026-07-05T16:48:23+00:00`
- project: `wf_0f586cf2-9eb`
- session_id: `c7679509-ee2b-4443-b240-0b3b1b1a7291`

Transcript

1. user (2026-06-21T14:42:18.604Z)

This is PR P0a from docs/PACKAGING_PLAN.md: introduce a resolved app-data HOME so a packaged app launched from an arbitrary CWD finds the same config/.env/DB/output, WITHOUT changing behaviour when RALLY_APP_HOME is unset. HARD CONTRACT: env unset => byte-identical to pre-P0a (CWD-relative). The repo: FastAPI engine, namespace package (no backend/__init__.py), hatchling editable install, Windows-first users, requires-python>=3.10. Verified facts: full suite 1221 passed/7 skipped, ruff+mypy clean, coverage 85.10%, and an e2e proof showed config+DB land under RALLY_APP_HOME (nothing leaks to CWD) and == CWD when unset.

Changed/new files in the worktree C:/Users/avidu/Projects/bhi-wt-p0a:
- NEW backend/utils/app_home.py (the resolver: app_home/app_path/default_config_path/default_env_path/resolve_output_dir/default_os_app_home)
- backend/config/loader.py (DEFAULT_CONFIG_PATH alias + use default_config_path())
- backend/main.py (load_dotenv prefers app-home .env; static mount package-relative; output_dir via resolve_output_dir at server-start + trim)
- scripts/doctor.py (writes config to default_config_path() not REPO_ROOT)
- NEW tests/test_app_home.py

LENS: test-gaps-and-doctor
Read tests/test_app_home.py + scripts/doctor.py + the diff. What behaviours are UNtested? (e.g. the main.py load_dotenv branch, the package-relative static mount, resolve_output_dir at the server-start/trim sites, doctor writing to default_config_path). Also: the doctor change moves the config write from REPO_ROOT to default_config_path() (CWD when env unset) ? is the behaviour change for "doctor run from a repo SUBDIR" acceptable/intended, and is it documented? Suggest the specific missing tests.

Read the actual files under C:/Users/avidu/Projects/bhi-wt-p0a. Return findings with severity blocker|major|minor|nit, each with file:line and a concrete fix. Be harsh but precise; do not invent issues.

2. assistant (2026-06-21T14:42:27.293Z)

I'll review this PR with the lens of test-gaps-and-doctor. Let me start by reading the actual files in the worktree.

3. user (2026-06-21T14:42:27.546Z)

```
1       """Resolved application-data HOME for a packaged, batteries-included install.
2
3       PACKAGING_PLAN.md P0a. Today every piece of mutable state ? ``config.json``, ``.env``,
4       the SQLite DB, ``output/`` ? is resolved RELATIVE TO THE CURRENT WORKING DIRECTORY, and
5       ``scripts/doctor.py`` writes ``config.json`` to the repo root while the server reads it
from
6       the CWD. Launched from an OS app menu (CWD ? repo root) those disagree and the user
silently
```

```
7       "loses" their index. This module introduces ONE resolver every state path roots on.
8
9       Design contract ? **DEFAULT IS BYTE-IDENTICAL TO TODAY**:
10        * ``RALLY_APP_HOME`` env set  ? that directory is the app home (the packaged launcher
sets it).
11        * ``RALLY_APP_HOME`` unset    ? ``Path(".")`` i.e. the CWD, exactly the pre-P0a
behaviour.
12
13      The OS-default location (``%APPDATA%`` / ``~/Library/Application Support`` /
``~/.local/share``)
14      is computed by :func:`default_os_app_home` but is **NOT** auto-selected here ? the
launcher (P2)
15      calls it and exports ``RALLY_APP_HOME`` explicitly. Keeping the resolver opt-in means
importing
16      this module has zero behavioural effect on the repo/dev/CI/test paths (no surprise
writes into a
17      real user profile during a test run).
18
19      stdlib-only (os, platform, pathlib) so it is safe to import at ``backend.main`` line 1 ?
before
20      the heavier pydantic-backed ``backend.config`` package ? keeping the early
``load_dotenv`` path light.
21      """
22
23      from __future__ import annotations
24
25      import os
26      import platform
27      from pathlib import Path
28
29      #: Environment variable the packaged launcher sets to pin the app-data home.
30      APP_HOME_ENV = "RALLY_APP_HOME"
31
32      #: OS-app-dir leaf used by :func:`default_os_app_home` (the product brand).
33      APP_DIR_NAME = "Khelsutra"
34
35
36      def app_home() -> Path:
37          """The resolved application-data home.
38
39          ``RALLY_APP_HOME`` (``~`` expanded) when set; otherwise ``Path(".")`` ? the CWD,
byte-identical
40          to the pre-P0a behaviour. Does NOT create the directory (callers create on write).
41          """
42          env = os.environ.get(APP_HOME_ENV)
43          if env and env.strip():
44              return Path(env.strip()).expanduser()
45          return Path(".")
46
47
48      def app_path(*parts: str | os.PathLike[str]) -> Path:
49          """A path under :func:`app_home` (e.g. ``app_path("output")``)."""
50          return app_home().joinpath(*[str(p) for p in parts])
51
52
53      def default_config_path() -> Path:
54          """``<app-home>/config.json`` ? the config file the loader reads and ``--setup``
writes.
55
56          With ``RALLY_APP_HOME`` unset this is ``Path("config.json")`` (CWD-relative),
unchanged.
57          """
58          return app_home() / "config.json"
59
60
```

```
61      def default_env_path() -> Path:
62          """``<app-home>/.env`` ? the dotenv file the server prefers (e.g.
``GEMINI_API_KEY``)."""
63          return app_home() / ".env"
64
65
66      def resolve_output_dir(output_dir: str) -> str:
67          """Root a RELATIVE ``output_dir`` under :func:`app_home`; pass an ABSOLUTE one
through.
68
69          With ``RALLY_APP_HOME`` unset, ``resolve_output_dir("output") == "output"`` (CWD-
relative),
70          byte-identical to today; with it set, ``"output"`` becomes ``<app-home>/output`` so
the SQLite
71          DB and produced reels live alongside the rest of the app's state.
72          """
73          if not output_dir:
74              return str(app_home() / "output")
75          if os.path.isabs(output_dir):
76              return output_dir
77          return str(app_home() / output_dir)
78
79
80      def default_os_app_home(app_name: str = APP_DIR_NAME) -> Path:
81          """The OS-conventional per-user app-data dir (for the packaged LAUNCHER to export as
82          ``RALLY_APP_HOME`` ? NOT auto-selected by :func:`app_home`).
83
84          * Windows ? ``%APPDATA%\\<app_name>`` (``~/AppData/Roaming/<app_name>`` if unset)
85          * macOS   ? ``~/Library/Application Support/<app_name>``
86          * Linux/other ? ``$XDG_DATA_HOME/<app_name>`` (``~/.local/share/<app_name>`` if
unset)
87          """
88          system = platform.system()
89          if system == "Windows":
90              base = os.environ.get("APPDATA") or str(Path.home() / "AppData" / "Roaming")
91          elif system == "Darwin":
92              base = str(Path.home() / "Library" / "Application Support")
93          else:
94              base = os.environ.get("XDG_DATA_HOME") or str(Path.home() / ".local" / "share")
95          return Path(base) / app_name
96
```

4. user (2026-06-21T14:42:27.816Z)

```
1       """Tests for the resolved application-data HOME (PACKAGING_PLAN.md P0a).
2
3       The load-bearing guarantee: with ``RALLY_APP_HOME`` UNSET, every resolver is byte-
identical to the
4       pre-P0a CWD-relative behaviour; with it SET, config/.env/output/DB all root under that
one home.
5       """
6
7       import os
8       from pathlib import Path
9
10      import pytest
11
12      from backend.utils import app_home as ah
13      from backend.config import loader
14
15
16      @pytest.fixture(autouse=True)
17      def _clear_app_home_env(monkeypatch):
18          """Each test starts with RALLY_APP_HOME unset unless it sets it explicitly."""
19          monkeypatch.delenv(ah.APP_HOME_ENV, raising=False)
```

```
20
21
22      # ---- default (env unset) is byte-identical to today ----
23
24      def test_app_home_defaults_to_cwd():
25          assert ah.app_home() == Path(".")
26
27
28      def test_default_config_path_unset_is_cwd_config_json():
29          # Pre-P0a this was the module constant Path("config.json").
30          assert ah.default_config_path() == Path("config.json")
31
32
33      def test_default_env_path_unset_is_cwd_dotenv():
34          assert ah.default_env_path() == Path(".env")
35
36
37      def test_resolve_output_dir_unset_is_identity_for_relative():
38          assert ah.resolve_output_dir("output") == "output"
39          assert ah.resolve_output_dir("custom") == "custom"
40
41
42      # ---- RALLY_APP_HOME set ? everything roots there ----
43
44      def test_app_home_honors_env(monkeypatch, tmp_path):
45          monkeypatch.setenv(ah.APP_HOME_ENV, str(tmp_path))
46          assert ah.app_home() == tmp_path
47
48
49      def test_app_home_expands_user(monkeypatch):
50          monkeypatch.setenv(ah.APP_HOME_ENV, "~/khelsutra-data")
51          assert ah.app_home() == Path.home() / "khelsutra-data"
52
53
54      def test_app_home_ignores_blank_env(monkeypatch):
55          monkeypatch.setenv(ah.APP_HOME_ENV, "   ")
56          assert ah.app_home() == Path(".")
57
58
59      def test_config_and_env_paths_root_under_home(monkeypatch, tmp_path):
60          monkeypatch.setenv(ah.APP_HOME_ENV, str(tmp_path))
61          assert ah.default_config_path() == tmp_path / "config.json"
62          assert ah.default_env_path() == tmp_path / ".env"
63          assert ah.app_path("output", "x.db") == tmp_path / "output" / "x.db"
64
65
66      def test_resolve_output_dir_set(monkeypatch, tmp_path):
67          monkeypatch.setenv(ah.APP_HOME_ENV, str(tmp_path))
68          assert ah.resolve_output_dir("output") == str(tmp_path / "output")
69
70
71      def test_resolve_output_dir_absolute_passthrough(monkeypatch, tmp_path):
72          monkeypatch.setenv(ah.APP_HOME_ENV, str(tmp_path))
73          abs_dir = str(tmp_path / "elsewhere") if os.path.isabs(str(tmp_path)) else
os.path.abspath("elsewhere")
74          assert ah.resolve_output_dir(abs_dir) == abs_dir
75
76
77      def test_resolve_output_dir_empty_falls_back_to_home_output(monkeypatch, tmp_path):
78          monkeypatch.setenv(ah.APP_HOME_ENV, str(tmp_path))
79          assert ah.resolve_output_dir("") == str(tmp_path / "output")
80
81
82      # ---- default_os_app_home: the OS-conventional dir the launcher exports (NOT auto-
selected) ----
```

```

    def test_default_os_app_home_windows(monkeypatch, tmp_path):
        monkeypatch.setattr(ah.platform, "system", lambda: "Windows")
        monkeypatch.setenv("APPDATA", str(tmp_path / "Roaming"))
        assert ah.default_os_app_home() == tmp_path / "Roaming" / "Khelsutra"


    def test_default_os_app_home_macos(monkeypatch):
        monkeypatch.setattr(ah.platform, "system", lambda: "Darwin")
        assert ah.default_os_app_home() == Path.home() / "Library" / "Application Support" /
"Khelsutra"


    def test_default_os_app_home_linux_xdg(monkeypatch, tmp_path):
        monkeypatch.setattr(ah.platform, "system", lambda: "Linux")
        monkeypatch.setenv("XDG_DATA_HOME", str(tmp_path / "share"))
        assert ah.default_os_app_home() == tmp_path / "share" / "Khelsutra"


    def test_default_os_app_home_linux_default(monkeypatch):
        monkeypatch.setattr(ah.platform, "system", lambda: "Linux")
        monkeypatch.delenv("XDG_DATA_HOME", raising=False)
        assert ah.default_os_app_home() == Path.home() / ".local" / "share" / "Khelsutra"


    # ---- loader integration: load_config() roots on the app-home ----

    def test_loader_reads_config_from_app_home(monkeypatch, tmp_path):
        monkeypatch.setenv(ah.APP_HOME_ENV, str(tmp_path))
        (tmp_path / "config.json").write_text('{"sport": "tennis"}', encoding="utf-8")
        cfg = loader.load_config()  # no explicit path ? default_config_path() under the
home
        assert cfg.sport == "tennis"


    def test_loader_no_config_in_home_uses_defaults(monkeypatch, tmp_path):
        monkeypatch.setenv(ah.APP_HOME_ENV, str(tmp_path))  # empty dir, no config.json
        cfg = loader.load_config()
        # Defaults load cleanly (no crash, no file) ? same contract as a missing CWD
config.json.
        assert cfg is not None
        assert hasattr(cfg, "indexing")


    def test_save_default_config_writes_under_app_home(monkeypatch, tmp_path):
        monkeypatch.setenv(ah.APP_HOME_ENV, str(tmp_path))
        written = loader.save_default_config()  # no explicit path
        assert written == tmp_path / "config.json"
        assert written.exists()


    def test_explicit_path_still_overrides_app_home(monkeypatch, tmp_path):
        monkeypatch.setenv(ah.APP_HOME_ENV, str(tmp_path / "home"))
        explicit = tmp_path / "explicit.json"
        explicit.write_text('{"sport": "pickleball"}', encoding="utf-8")
        cfg = loader.load_config(str(explicit))
        assert cfg.sport == "pickleball"

```

5. user (2026-06-21T14:42:28.111Z)

```
import sys
import os
import subprocess
import shutil
```

```
5
6       # This diagnostics/bootstrap script lives in scripts/ but operates on the repo
7       # root (requirements.txt, config.json). Resolve the root from this file's
8       # location so it works regardless of the caller's cwd.
9       REPO_ROOT = os.path.dirname(os.path.dirname(os.path.abspath(__file__)))
10
11      def check_command(cmd):
12          """Check if a system command is available."""
13          return shutil.which(cmd) is not None
14
15      def run_pip_install():
16          """Install required Python packages."""
17          print("Installing Python packages from requirements.txt...")
18
19          # Check if NVIDIA GPU is present
20          has_gpu = check_command("nvidia-smi")
21
22          requirements_path = os.path.join(REPO_ROOT, "requirements.txt")
23          pip_cmd = [sys.executable, "-m", "pip", "install", "--user", "-r",
requirements_path]
24
25          import platform
26          if has_gpu:
27              print("[INFO] NVIDIA GPU detected. Configuring PyTorch to download CUDA-enabled
wheels.")
28              pip_cmd.extend(["--extra-index-url", "https://download.pytorch.org/whl/cu124"])
29          elif platform.system() == "Darwin":
30              # macOS has no CUDA; the default PyPI wheel already ships CPU + Apple MPS.
31              print("[INFO] macOS detected. Using default PyTorch wheels (CPU + Apple MPS).")
32          else:
33              print("[INFO] No NVIDIA GPU detected. Defaulting to PyTorch CPU wheels.")
34              pip_cmd.extend(["--extra-index-url", "https://download.pytorch.org/whl/cpu"])
35
36          try:
37              subprocess.check_call(pip_cmd)
38              print("[SUCCESS] Python packages installed.")
39              return True
40          except subprocess.CalledProcessError as e:
41              print(f"[ERROR] Failed to install Python packages: {e}")
42              return False
43
44      def init_default_config():
45          """Initialize a default config.json if it doesn't exist.
46
47          Now delegates to the canonical backend.config loader so that defaults
48          stay in sync with the Pydantic models (single source of truth).
49          """
50          try:
51              from backend.config import save_default_config
52              from backend.utils.app_home import default_config_path
53          except Exception as e:
54              print(f"[ERROR] Could not import backend.config loader: {e}")
55              print("[INFO] Falling back to no-op for config initialization.")
56              return
57
58          # PACKAGING_PLAN.md P0a: write to the SAME app-home config path the server READS
59          # (RALLY_APP_HOME, else CWD/config.json) ? previously this wrote to the repo root
while
60          # the server read from the CWD, so --setup and the server disagreed once CWD ? repo
root.
61          config_path = str(default_config_path())
62          if os.path.exists(config_path):
63              print(f"[INFO] {config_path} already exists. Skipping initialization.")
64              return
65
```

```
66          try:
67              saved_path = save_default_config(config_path)
68              print(f"[SUCCESS] config.json initialized with default settings at
{saved_path}.")
69          except Exception as e:
70              print(f"[ERROR] Failed to create config.json via loader: {e}")
71
72      def run_diagnostics():
73          """Perform system-wide checks for FFmpeg and PyTorch/CUDA."""
74          print("=" * 50)
75          print("SPORTS HIGHLIGHT INDEXER SETUP DIAGNOSTICS")
76          print("=" * 50)
77
78          # 1. Check Python Version
79          print(f"Python Version:
{sys.version_info.major}.{sys.version_info.minor}.{sys.version_info.micro}")
80          if sys.version_info.major < 3 or (sys.version_info.major == 3 and
sys.version_info.minor < 8):
81              print("[WARNING] Python 3.8+ is highly recommended!")
82          else:
83              print("[PASS] Python version check passed.")
84
85          print("-" * 50)
86
87          # 2. Check FFmpeg / FFprobe
88          ffmpeg_ok = check_command("ffmpeg")
89          ffprobe_ok = check_command("ffprobe")
90
91          if ffmpeg_ok:
92              print("[PASS] FFmpeg is installed and available in PATH.")
93          else:
94              print("[FAIL] FFmpeg is NOT found in your PATH.")
95
96          if ffprobe_ok:
97              print("[PASS] FFprobe is installed and available in PATH.")
98          else:
99              print("[FAIL] FFprobe is NOT found in your PATH.")
100
101         if not (ffmpeg_ok and ffprobe_ok):
102             import platform
103             print("\n" + "!" * 50)
104             print("CRITICAL: FFmpeg and FFprobe are REQUIRED to run this application.")
105             _sys = platform.system()
106             if _sys == "Windows":
107                 print("Install on Windows:  winget install Gyan.FFmpeg")
108             elif _sys == "Darwin":
109                 print("Install on macOS:    brew install ffmpeg")
110             else:
111                 print("Install on Linux:    sudo apt-get install -y ffmpeg   (or your
distro's package manager)")
112             print("Or download a build from https://ffmpeg.org/download.html and put
ffmpeg/ffprobe on PATH.")
113             print("!" * 50 + "\n")
114         else:
115             print("[SUCCESS] All essential media command line tools are ready.")
116
117         print("-" * 50)
118
119         # 3. Check ML / GPU (Optional but good)
120         try:
121             import torch
122             cuda_ok = torch.cuda.is_available()
123             if cuda_ok:
124                 print(f"[PASS] PyTorch loaded successfully. CUDA (GPU) is AVAILABLE:
{torch.cuda.get_device_name(0)}")
```

```
125             else:
126                 print("[INFO] PyTorch loaded successfully, but CUDA (GPU) is NOT available.
Falling back to CPU.")
127         except ImportError:
128             print("[INFO] PyTorch is not installed. Relevance checks will run using
lightweight visual heuristics.")
129
130         print("=" * 50)
131         return ffmpeg_ok and ffprobe_ok
132
133     if __name__ == "__main__":
134         init_default_config()
135         media_ok = run_diagnostics()
136         run_pip_install()
137         print("\n[INFO] Setup complete. If diagnostics passed, you can now run the app!")
138
```

6. assistant (2026-06-21T14:42:31.166Z)

Let me read the remaining changed files: `backend/main.py` and `backend/config/loader.py`, plus the diff and the packaging plan.

7. user (2026-06-21T14:42:32.612Z)

```
1       """Configuration loader and saver.
2
3       Provides a single place to load validated AppConfig with built-in defaults
4       merged with optional file overrides (config.json).
5
6       This replaces scattered json.load + .get() patterns throughout the codebase.
7       """
8
9       from __future__ import annotations
10
11      import json
12      import logging
13      import os
14      from pathlib import Path
15      from typing import Any
16
17      from pydantic import BaseModel
18
19      from backend.utils.app_home import default_config_path
20
21      from .models import AppConfig
22      from .presets import (
23          COMPUTE_TARGET_FOR_PROFILE,
24          PROFILE_PRESETS,
25          is_known_profile,
26          preset_for,
27          suggest_profile,
28      )
29
30      logger = logging.getLogger(__name__)
31
32      # Back-compat alias (was a module constant; no external consumers). Prefer
33      # ``default_config_path()`` ? it honours ``RALLY_APP_HOME`` (PACKAGING_PLAN.md P0a).
With the
34      # env unset this is ``Path("config.json")`` (CWD-relative), exactly the pre-P0a value.
35      DEFAULT_CONFIG_PATH = default_config_path()
36
37
38      def _deep_merge(base: dict[str, Any], override: dict[str, Any]) -> dict[str, Any]:
39          """Recursively overlay ``override`` onto ``base`` (override wins; dicts merge,
scalars
```

```
        replace). Used ONLY for the profile-preset precedence path so a config.json sub-key
        overrides a preset sub-key while sibling preset/default sub-keys survive ? the no-
profile
        path keeps the original shallow ``{**defaults, **file_data}`` merge unchanged."""
        out = dict(base)
        for key, val in override.items():
            if isinstance(val, dict) and isinstance(out.get(key), dict):
                out[key] = _deep_merge(out[key], val)
            else:
                out[key] = val
        return out


    def _resolve_explicit_profile(file_data: dict[str, Any]) -> str:
        """The EXPLICITLY chosen deployment profile, env winning over config.json. ``""`` if
none
        is set. Auto-detect is deliberately NOT consulted here ? it only suggests (D15), so
a bare
        environment never silently selects a profile (and never silently spends on cloud).

        An *unrecognised* ``DEPLOYMENT_PROFILE`` env value warns and FALLS THROUGH to the
file's
        profile (rather than suppressing it) ? so an env typo can't leave the recorded
profile
        asserting a preset that was never applied. A bad value in config.json is left to
surface as
        a hard, typed-Literal error downstream (config-file correctness)."""
        env_profile = os.environ.get("DEPLOYMENT_PROFILE")
        if env_profile and env_profile.strip():
            ep = env_profile.strip()
            if is_known_profile(ep):
                return ep
            logger.warning(
                "[CONFIG] unrecognized DEPLOYMENT_PROFILE env value %r (valid: %s); ignoring
it "
                "and falling back to config.json deployment.profile.", ep, ",
".join(PROFILE_PRESETS),
            )
            # fall through to the file's profile (env typo must not suppress a valid file
choice)
        dep = file_data.get("deployment")
        if isinstance(dep, dict):
            prof = dep.get("profile")
            if isinstance(prof, str) and prof.strip():
                return prof.strip()
        return ""


    def _unknown_key_paths(data: dict[str, Any], model_cls: type[BaseModel], prefix: str =
"") -> list[str]:
        """Dotted paths in `data` that the typed config will not honor.

        Two silent failure modes exist without this check: a typo'd TOP-LEVEL key is
        kept by AppConfig's ``extra="allow"`` but never read by typed code, and a
        typo'd key INSIDE a section is silently dropped by Pydantic. Either way the
        user's intent is lost ? collect every such key so load_config can warn.
        """
        unknown: list[str] = []
        fields = model_cls.model_fields
        for key, val in data.items():
            if key not in fields:
                unknown.append(prefix + key)
                continue
            ann = fields[key].annotation
            if isinstance(val, dict) and isinstance(ann, type) and issubclass(ann,
```

```
BaseModel):
                unknown.extend(_unknown_key_paths(val, ann, f"{prefix}{key}."))
        return unknown


    def _get_builtin_defaults() -> dict[str, Any]:
        """Return a plain dict of the current built-in defaults.

        The AppConfig model is the single source of truth for defaults.
        """
        return AppConfig().model_dump(mode="json")


    def load_config(path: str | Path | None = None) -> AppConfig:
        """Load configuration with the following precedence:

        1. Built-in defaults defined in the Pydantic models.
        2. Values from the JSON file (if present and readable).
        3. (Future) environment / CLI overrides.

        Missing file or unreadable file ? returns a fully defaulted AppConfig.
        Unknown keys in the file are tolerated (extra="allow" on the model).
        """
        cfg_path = Path(path) if path is not None else default_config_path()

        file_data: dict[str, Any] = {}
        if cfg_path.exists():
            try:
                with open(cfg_path, "r", encoding="utf-8") as f:
                    file_data = json.load(f)
            except Exception as exc:
                # Non-fatal: continue with defaults + whatever we could parse.
                # In production we might log here.
                logger.warning(f"Failed to read {cfg_path}: {exc}. Using defaults + partial
data.")

        # A config.json that is valid JSON but not an OBJECT (e.g. `[]`, `42`, `"x"`,
`null`) would
        # otherwise crash startup: a truthy non-mapping hits `.items()` in
_unknown_key_paths, and any
        # non-mapping breaks the `{**defaults, **file_data}` unpack. Degrade to defaults +
warn, per the
        # loader's "bad input is non-fatal" contract.
        if not isinstance(file_data, dict):
            logger.warning("%s is not a JSON object (got %s); ignoring it and using
defaults.",
                           cfg_path, type(file_data).__name__)
            file_data = {}

        if file_data:
            unknown = _unknown_key_paths(file_data, AppConfig)
            if unknown:
                logger.warning(
                    "[CONFIG WARNING] %s contains keys the typed config does not "
                    "recognize (typo'd or removed knobs ? top-level extras are kept "
                    "but unread; unknown section keys are silently dropped): %s",
                    cfg_path, ", ".join(sorted(unknown)),
                )

        # Precedence: built-in defaults < profile preset < config.json (< env/--set, applied
at
        # consumption sites downstream). A *deployment profile* is opt-in ? it is applied
ONLY when
        # explicitly selected (DEPLOYMENT_PROFILE env or config.json deployment.profile).
With no
```

```
        # profile the merge is byte-identical to the pre-M1 loader, so the default path is
unchanged.
        defaults = _get_builtin_defaults()
        profile = _resolve_explicit_profile(file_data)

        if profile and not is_known_profile(profile):
            # Only an unrecognised value from config.json reaches here (env typos already
fell back
            # in _resolve_explicit_profile). Warn + apply no preset; the bad value also hits
the
            # typed Literal at model_validate below, a hard clear error ? loud, never
silent.
            logger.warning(
                "[CONFIG] unrecognized deployment profile %r (valid: %s); applying no
preset.",
                profile, ", ".join(PROFILE_PRESETS),
            )
            profile = ""

        if profile:
            merged = _deep_merge(_deep_merge(defaults, preset_for(profile)), file_data)
            # Record the profile actually applied ? the env may have chosen it over
config.json,
            # so re-stamp it onto the merged deployment section to keep the record honest.
            merged.setdefault("deployment", {})
            if isinstance(merged["deployment"], dict):
                merged["deployment"]["profile"] = profile
                ct = merged["deployment"].get("compute_target")
                canonical = COMPUTE_TARGET_FOR_PROFILE.get(profile)
                if not ct and canonical:
                    # An active profile with no explicit compute_target (e.g. config.json
blanked it
                    # to "") gets its canonical target back ? the record never silently lies
about an
                    # active profile's compute_target (it stays meaningful for the M2+
ComputeBackend).
                    merged["deployment"]["compute_target"] = canonical
                elif ct and canonical and ct != canonical:
                    logger.warning(
                        "[CONFIG] deployment profile %r normally uses compute_target=%r but
"
                        "config.json overrides it to %r ? honoring the explicit override.",
                        profile, canonical, ct,
                    )
                logger.info("[CONFIG] deployment profile ACTIVE: %s (compute_target=%s)",
                            profile, merged["deployment"].get("compute_target"))
        else:
            # No profile ? the original shallow merge (Pydantic re-defaults nested
sections).
            merged = {**defaults, **file_data}
            suggestion = suggest_profile()  # cheap, env-only (no torch); SUGGESTS only
(D15)
            if suggestion:
                logger.debug(
                    "[CONFIG] no deployment.profile set (manual knobs). Environment suggests
%r ? "
                    "set deployment.profile or DEPLOYMENT_PROFILE to apply (auto-detect
never "
                    "auto-applies ? D15).", suggestion,
                )

        return AppConfig.model_validate(merged)


    def save_default_config(path: str | Path | None = None) -> Path:
```

```
202         """Write a fresh default config.json.
203
204         Used by scripts/doctor.py diagnostics and the --setup CLI path.
205         Overwrites if the file exists.
206         """
207         cfg_path = Path(path) if path is not None else default_config_path()
208         cfg = AppConfig()
209         cfg_path.parent.mkdir(parents=True, exist_ok=True)
210         cfg_path.write_text(cfg.model_dump_json(indent=4), encoding="utf-8")
211         return cfg_path
212
213
214     # Backwards-compatible alias used by some older call sites during transition.
215     def get_default_config() -> dict[str, Any]:
216         """Return defaults as a plain dict (for transitional use only)."""
217         return _get_builtin_defaults()
218
```

8. user (2026-06-21T14:42:33.100Z)

```
1       import os
2       import sys
3       import argparse
4       import logging
5       import uuid
6       from urllib.parse import quote
7       import uvicorn
8       from dotenv import load_dotenv
9
10      # PACKAGING_PLAN.md P0a: prefer an app-home .env (the packaged launcher pins
RALLY_APP_HOME),
11      # else fall back to the historical CWD+walk-up search so the repo/dev path is unchanged.
12      # stdlib-only import ? keeps this pre-pydantic line light.
13      from backend.utils.app_home import default_env_path, resolve_output_dir
14
15      _env_path = default_env_path()
16      if _env_path.exists():
17          load_dotenv(_env_path)
18      else:
19          load_dotenv()
20
21      # Centralized logging setup for the indexer (hot paths use logger, CLI output uses print
for direct user-facing messages).
22      logging.basicConfig(
23          level=logging.INFO,
24          format='%(asctime)s [%(levelname)s] %(name)s: %(message)s',
25          datefmt='%H:%M:%S'
26      )
27      logger = logging.getLogger(__name__)
28      from fastapi import FastAPI, HTTPException, Request
29      from fastapi.middleware.cors import CORSMiddleware
30      from pydantic import BaseModel
31      from typing import List, Dict, Any, Optional
32
33      from fastapi.staticfiles import StaticFiles
34      from fastapi.responses import FileResponse, HTMLResponse, JSONResponse
35      from backend.pipeline.validators.base import registry
36      from backend.infrastructure.database import Database
37      from backend.pipeline.segmenters.base import segmenter_registry
38      from backend.pipeline.stitcher.compiler import VideoCompiler
39      from backend.utils.hashing import compute_video_id  # canonical file-identity hashing
40      from backend.utils.paths import resolve_under_root, safe_output_filename,
validate_input_path
41      from backend.config import (  # new typed config
42          load_config as load_app_config,
```

```
43          AppConfig,
44          StitchingConfig,
45          enforce_startup_checks,
46          StartupCheckError,
47      )
48      from backend.results import Failure  # standardized error/result contract
49      from backend.reporting import emit_indexer_report
50      from backend import storage  # M2: URI-aware input acquisition (local = identity
passthrough)
51      from backend import billing  # M8: per-job cost ledger (USD internal / INR display)
52      from backend.api import auth as edge_auth  # M9: Cloudflare Access edge-auth (default-
OFF)
53
54      app = FastAPI(title="Sports Highlight Indexer API")
55
56
57      def _cors_origins_from_env(env: Optional[Dict[str, str]] = None) -> List[str]:
58          """M9: CORS allow-origins from the ``RALLY_CORS_ALLOW_ORIGINS`` env var (comma-
separated),
59          default ``["*"]``. Read from the ENV (not a cwd ``config.json``) so importing
``backend.main``
60          does zero filesystem I/O ? a hosted/multi-tenant deploy sets this to lock CORS to
its origin."""
61          src = os.environ if env is None else env
62          raw = (src.get("RALLY_CORS_ALLOW_ORIGINS", "") or "").strip()
63          origins = [o.strip() for o in raw.split(",") if o.strip()]
64          return origins or ["*"]
65
66
67      # CORS for the web UI. allow_origins='*' with allow_credentials=True is rejected by
browsers per the
68      # fetch spec + is unsafe to carry into multi-tenant; this tool uses no cookies, so
credentials stay
69      # off. The origin list is configurable via the env var above (default ["*"]) so a hosted
deploy can
70      # lock it to its origin; the middleware is fixed at startup.
71      app.add_middleware(
72          CORSMiddleware,
73          allow_origins=_cors_origins_from_env(),
74          allow_credentials=False,
75          allow_methods=["*"],
76          allow_headers=["*"],
77      )
78
79      # Serves static frontend files directly (now under api/ per approved structure).
80      # PACKAGING_PLAN.md P0a: resolve PACKAGE-relative (not CWD-relative) so launching from
an
81      # arbitrary directory finds the shipped assets instead of creating a stray
./backend/api/static.
82      # (The legacy in-engine dashboard; the real SPA gets bundled here in P1a.)
83      _STATIC_DIR = os.path.join(os.path.dirname(os.path.abspath(__file__)), "api", "static")
84      os.makedirs(_STATIC_DIR, exist_ok=True)
85      app.mount("/static", StaticFiles(directory=_STATIC_DIR), name="static")
86
87      @app.get("/", response_class=HTMLResponse)
88      def serve_index():
89          index_path = os.path.join(_STATIC_DIR, "index.html")
90          if os.path.exists(index_path):
91              with open(index_path, "r", encoding="utf-8") as f:
92                  return f.read()
93          return "<h3>Sports Highlight Indexer Server is Running. Please create index.html in
backend/api/static/</h3>"
94
95      # Global variables initialized at startup
96      db_instance: Optional[Database] = None
```

```
    global_config: Optional[AppConfig] = None  # now typed (Pydantic model)

    # Async job worker ? drain/wake loop (DEFERRED_HARDENING_PLAN #16, EXECUTION_POLICY.md P3)
    # now lives in backend.api.job_worker. Re-exported here so the names stay addressable on
    # `backend.main` (`JobWaiting`, `_drain_due_jobs`, `_arm_wake_timer`, `_get_executor`,
    # `_run_claimed_job`, `_MAX_DRAIN_WAKE_SEC`, ...). The worker resolves these seams THROUGH
    # this module object at call-time, so `monkeypatch.setattr(backend.main, ...)` still
    # intercepts the inter-function calls (a parked job arms the PATCHED `_arm_wake_timer`,
    # so no real daemon timer leaks in tests). Behavior is unchanged by the extraction.
    from backend.api.job_worker import (  # noqa: F401  (re-exports: names stay addressable on backend.main)
        JobWaiting,
        _arm_wake_timer,
        _drain_due_jobs,
        _get_executor,
        _job_to_request,
        _MAX_DRAIN_WAKE_SEC,
        _maybe_record_job_cost,
        _run_claimed_job,
        _schedule_next_drain_if_waiting,
    )

    # Legacy thin wrapper for any remaining direct calls during transition.
    # New code should import load_app_config / AppConfig from backend.config directly.
    def load_config(config_path: str = "config.json") -> Dict[str, Any]:
        cfg = load_app_config(config_path)
        return cfg.model_dump(mode="json")

    # --- API Models ---
    class ProcessRequest(BaseModel):
        video_path: str
        player_pool: Optional[List[str]] = None
        max_frames: Optional[int] = None
        segmenter: Optional[str] = None

    class QueryRequest(BaseModel):
        video_id: str
        server: Optional[str] = None
        receiver: Optional[str] = None
        duration_min: Optional[float] = None
        event_type: Optional[str] = None

    class CompileRequest(BaseModel):
        video_id: str
        segment_ids: List[int]
        output_filename: str

    def _finalize_reingest(video_id: str, segments, play_wm: int, cand_wm: int) -> None:
        """Commit a (re)segmentation with destroy-AFTER-confirm semantics.

        On a non-empty run, drop the PRIOR rows (id <= the pre-run watermark) so only the
        fresh segments remain. On an empty run, drop the NEW partial rows (id > watermark) and
        keep the prior good results intact ? a failed/empty re-run never destroys prior data.
        """
        if segments:
            db_instance.prune_segments(video_id, play_upto=play_wm, cand_upto=cand_wm)
        else:
            db_instance.prune_segments(video_id, play_after=play_wm, cand_after=cand_wm)


    # --- API Endpoints ---
```

```
    @app.get("/api/config")
    def get_config():
        return global_config

    @app.get("/api/health")
    def health():
        """Cheap liveness/readiness probe (no auth) for the SPA's offline banner
        (HOSTING_PLAN.md:42). Reports the configured sport + default segmenter so the UI can
        confirm the engine is reachable and which detector is wired. Never raises."""
        sport = getattr(global_config, "sport", None)
        indexing = getattr(global_config, "indexing", None)
        return {
            "status": "ok",
            "sport": sport,
            "default_segmenter": getattr(indexing, "default_segmenter", None),
        }

    def _execute_processing(req: ProcessRequest, progress=None) -> Dict[str, Any]:
        """The full hash ? validate ? segment ? report pipeline for one video.

        This is the former synchronous /api/process body, unchanged in behavior, now run on
        the job worker thread (DEFERRED_HARDENING_PLAN #16). Returns the same response dict
        the sync endpoint used to return (UI compatibility); the per-video observability
        report is still emitted in `finally:` and grafted as response["reports"].

        `progress` is an optional callable(stage: str) used to surface coarse job progress.
        Raises on unexpected internal errors ? the caller records those as a job failure
        (after this function has already restored the pre-run segments, see below).
        """
        notify = progress or (lambda stage: None)

        notify("hashing")
        # M2: resolve the input to a LOCAL path. A bare/local path is returned unchanged
(identity ?
        # byte-identical to today); a bucket/Drive URI is fetched to scratch first. The
original
        # req.video_path (the source URI) is kept for the DB record + report; only the file-
reading
        # consumers (hash / validators / segmenter) use the resolved local path.
        local_video = storage.acquire_local_input(req.video_path, getattr(global_config,
"storage", None))
        video_id = compute_video_id(local_video)
        was_reingest = db_instance.get_video(video_id) is not None

        # 1. Run pluggable validators (with-context variant surfaces ffprobe_meta for the
report)
        notify("validating")
        logger.info(f"Running validations for {req.video_path}...")
        val_results, val_context = registry.run_all_with_context(local_video, global_config)
        failures = [r for r in val_results if not r.passed]

        # typed AppConfig: attribute access for the default segmenter (with safe fallback)
        default_seg = getattr(getattr(global_config, "indexing", None), "default_segmenter",
"motion")
        seg_name = req.segmenter or default_seg
        segmenter_actual = None
        segmentation_ran = False
        response: Optional[Dict[str, Any]] = None
        try:
            if failures:
                # add_video is an UPSERT (no cascade) ? a failed re-validation only updates
                # status; it no longer destroys the video's prior good segments.
                db_instance.add_video(video_id, req.video_path, "failed", [r.model_dump()
for r in val_results])
                response = {
```

```
                    "video_id": video_id,
                    "status": "failed",
                    "message": "Video failed validation checks.",
                    "results": [r.model_dump() for r in val_results],
                }
                return response

            # 2. Run segmenter & indexer
            logger.info("[API] Video passed checks. Segmenting and indexing...")
            db_instance.add_video(video_id, req.video_path, "processed", [r.model_dump() for
r in val_results])

            segmenter_cls = segmenter_registry.get(seg_name)
            if not segmenter_cls:
                # Submit-time validation already 400s unknown names; this is the defensive
                # in-worker path (e.g. config default pointing at an unregistered
segmenter).
                available = list(segmenter_registry.list_available().keys())
                response = {
                    "video_id": video_id,
                    "status": "failed",
                    "message": f"Segmenter '{seg_name}' not found. Available segmenters:
{available}",
                    "results": [r.model_dump() for r in val_results],
                    "play_segments": [],
                }
                return response

            logger.info(f"[API] Using segmenter: {seg_name}")
            notify(f"segmenting ({seg_name})")
            segmenter_actual = seg_name
            # Destroy-AFTER-confirm: remember the pre-run rows, then keep new + drop old
only
            # if the run yields segments; otherwise drop the new partial rows and keep the
old.
            play_wm, cand_wm = db_instance.segment_watermarks(video_id)
            segmenter = segmenter_cls(db_instance, global_config)
            try:
                result = segmenter.process_video(video_id, local_video, req.player_pool,
req.max_frames)
            except Exception:
                # A raising segmenter must not leave half-written rows: restore the pre-run
                # state (same destroy-after-confirm contract as the Failure branch), then
let
                # the job worker record the failure.
                db_instance.prune_segments(video_id, play_after=play_wm, cand_after=cand_wm)
                raise
            segmentation_ran = True

            if isinstance(result, Failure):
                logger.error(f"[API] Segmenter failed: {result.message}")
                db_instance.prune_segments(video_id, play_after=play_wm, cand_after=cand_wm)
                response = {
                    "video_id": video_id,
                    "status": "failed",
                    "message": f"Segmenter '{seg_name}' failed: {result.message}",
                    "results": [r.model_dump() for r in val_results],
                    "play_segments": [],
                }
                return response

            segments = result.value if hasattr(result, "value") else result
            notify("finalizing")
            _finalize_reingest(video_id, segments, play_wm, cand_wm)

```

```
            response = {
                "video_id": video_id,
                "status": "success",
                "message": f"Successfully indexed {len(segments)} play segments using
'{seg_name}' segmenter.",
                "results": [r.model_dump() for r in val_results],
                "play_segments": segments,
            }
            return response
        finally:
            # Always emit the per-video observability report (next to the video, or to
            # report.output_dir). Never raises. Surface the paths in the response.
            paths = emit_indexer_report(
                db=db_instance, video_id=video_id, video_path=req.video_path,
config=global_config,
                val_results=val_results, val_context=val_context,
                segmenter_requested=req.segmenter, segmenter_actual=segmenter_actual,
                was_reingest=was_reingest, segmentation_ran=segmentation_ran,
            )
            if paths and isinstance(response, dict):
                response["reports"] = paths


    @app.post("/api/process", status_code=202)
    def process_video(req: ProcessRequest, request: Request):
        """Submit a processing job (202 + job_id). Poll GET /api/jobs/{job_id} for state.

        Fast-fail checks (bad path, missing file, unknown segmenter) happen here so the
        client gets an immediate 4xx; everything slow ? including MD5 hashing of the file ?
        runs on the worker (DEFERRED_HARDENING_PLAN #16). The row is created 'queued'; the
        worker drains it via `claim_due_job` (EXECUTION_POLICY.md P3 self-scheduling loop).
        """
        # M9 (D9): edge-auth tenant resolution. DEFAULT-OFF ? owner_id None (the local
single-user,
        # unchanged). When edge_auth_enabled, the Cf-Access JWT is required + verified (a
missing or
        # spoofed identity header fails here with 401, before any work) and tags the job's
owner_id.
        try:
            owner_id = edge_auth.resolve_tenant(request.headers, global_config)
        except edge_auth.EdgeAuthError as e:
            raise HTTPException(status_code=401, detail=f"edge auth: {e}") from e

        allowed_root = getattr(getattr(global_config, "security", None),
"allowed_video_root", None)
        try:
            validate_input_path(req.video_path, allowed_root=allowed_root)
            # M2: resolve the input once (path or storage URI) ? raises ValueError for an
unsupported
            # scheme, which we map to a clean 400 (an unknown scheme must never 500).
            input_ref = storage.resolve_input_ref(req.video_path, getattr(global_config,
"storage", None))
        except ValueError as e:
            raise HTTPException(status_code=400, detail=str(e)) from e
        # The local-existence fast-fail applies only to a LOCAL input, and stats the
RESOLVED local path
        # (input_ref.key ? e.g. local://? stripped to the bare path), not the raw URI
string. A
        # bucket/Drive URI can't be cheaply os.path.exists()'d ? its existence is verified
when the worker
        # fetches it (a fetch miss fails the job loudly). validate_input_path above still
runs for every
        # input: its null-byte guard always applies, and a configured allowed_video_root
(hosted mode)
        # rejects a non-local URI as out-of-root.
```

```
        if input_ref.backend == "local" and not os.path.exists(input_ref.key):
            raise HTTPException(status_code=404, detail=f"Video file not found at
'{req.video_path}'")
        if req.segmenter and not segmenter_registry.get(req.segmenter):
            available = list(segmenter_registry.list_available().keys())
            raise HTTPException(
                status_code=400,
                detail=f"Segmenter '{req.segmenter}' not found. Available segmenters:
{available}"
            )

        job_id = uuid.uuid4().hex
        if not db_instance.create_job(job_id, req.video_path, req.segmenter,
                                      req.player_pool, req.max_frames, owner_id=owner_id):
            raise HTTPException(status_code=500, detail="Failed to persist job record.")
        _get_executor().submit(_drain_due_jobs)
        return JSONResponse(
            status_code=202,
            content={"job_id": job_id, "status": "queued", "poll": f"/api/jobs/{job_id}"},
        )


    @app.get("/api/jobs/{job_id}/cost")
    def get_job_cost(job_id: str):
        """Per-job cost (COMPUTE_DECOUPLED_SERVING M8, D8). ``cost_usd`` is the source of
truth (matches
        GCP billing); ``cost_inr`` is a display conversion at the configured FX rate.
``cost_usd`` is
        null until billing records it (default-OFF / the placeholder pricebook ? 0)."""
        cost = db_instance.get_job_cost(job_id)
        if cost is None:
            raise HTTPException(status_code=404, detail=f"Unknown job_id '{job_id}'")
        fx = global_config.billing.fx_inr_per_usd
        usd = cost.get("cost_usd")
        cost["cost_inr"] = billing.to_inr(usd, fx) if usd is not None else None
        cost["fx_inr_per_usd"] = fx
        return cost


    @app.get("/api/videos/{video_id}/cost")
    def get_video_cost(video_id: str):
        """Aggregate per-video cost across its jobs (M8): ``total_cost_usd`` +
``total_cost_inr`` + the
        per-job rows. A video is 1:1 with an infer job, but a re-ingest can produce
several."""
        agg = db_instance.get_video_cost(video_id)
        fx = global_config.billing.fx_inr_per_usd
        agg["total_cost_inr"] = billing.to_inr(agg["total_cost_usd"], fx)
        agg["fx_inr_per_usd"] = fx
        return agg


    @app.get("/api/jobs/{job_id}")
    def get_job(job_id: str):
        """Job state: {status, progress, result, reports}. `status` = execution state
        (queued|running|done|failed); the pipeline verdict is result["status"]."""
        job = db_instance.get_job(job_id)
        if not job:
            raise HTTPException(status_code=404, detail=f"Unknown job_id '{job_id}'")
        result = job.get("result")
        return {
            "job_id": job["id"],
            "status": job["status"],
            "progress": job.get("progress"),
            "video_id": job.get("video_id"),
```

```
            "error": job.get("error"),
            "result": result,
            "reports": (result or {}).get("reports"),
            "created_at": job.get("created_at"),
            "started_at": job.get("started_at"),
            "finished_at": job.get("finished_at"),
        }

    # --- Chunked upload (B2, PR-2) ----------------------------------------------------
    # The chunked-upload island (in-process registry + helpers + the 4 endpoints) now lives
in
    # backend.api.uploads as a self-contained APIRouter. It resolves the live startup config
via
    # backend.main.global_config, so behavior ? route paths, status codes, body-size limits,
    # sequential-part logic, and abort cleanup ? is unchanged.
    from backend.api import uploads as uploads_api

    app.include_router(uploads_api.router)


    # --- Highlight download (B3, PR-2) ------------------------------------------------

    @app.get("/api/highlights/{video_id}/{filename}")
    def download_highlight(video_id: str, filename: str):
        """Stream a compiled highlight reel ? `filename` is the bare name given to
/api/compile
        (which only writes into output_dir; the browser previously got back an unreachable
        server-local path)."""
        try:
            name = safe_output_filename(filename)
        except ValueError as e:
            raise HTTPException(status_code=400, detail=str(e)) from e
        if not db_instance.get_video(video_id):
            raise HTTPException(status_code=404, detail="Indexed video record not found.")
        output_dir = getattr(getattr(global_config, "output", None), "output_dir", "output")
or "output"
        path = os.path.join(output_dir, name)
        try:
            path = resolve_under_root(path, output_dir)
        except ValueError as e:
            raise HTTPException(status_code=400, detail=str(e)) from e
        if not os.path.isfile(path):
            raise HTTPException(status_code=404,
                                detail=f"No compiled file named '{name}'. Compile first via
/api/compile.")
        media = "video/quicktime" if name.lower().endswith(".mov") else "video/mp4"
        return FileResponse(path, media_type=media, filename=name)


    @app.post("/api/query")
    def query_play_segments(req: QueryRequest):
        filters = {
            "server": req.server,
            "receiver": req.receiver,
            "duration_min": req.duration_min,
            "event_type": req.event_type
        }
        segments = db_instance.query_play_segments(req.video_id, filters)
        return {"play_segments": segments}

    @app.post("/api/compile")
    def compile_highlights(req: CompileRequest):
        video = db_instance.get_video(req.video_id)
        if not video:
            raise HTTPException(status_code=404, detail="Indexed video record not found.")
```

```
444
445         # Retrieve matching play segments from db
446         all_segments = db_instance.query_play_segments(req.video_id, {})
447         matched_segments = [s for s in all_segments if s["id"] in req.segment_ids]
448
449         if not matched_segments:
450             raise HTTPException(status_code=400, detail="No matching play segments found to
stitch.")
451
452         # Reject path traversal / absolute output names (os.path.join would otherwise let an
453         # absolute or '..'-laden output_filename write anywhere on disk).
454         try:
455             out_name = safe_output_filename(req.output_filename)
456         except ValueError as e:
457             raise HTTPException(status_code=400, detail=str(e)) from e
458
459         # The configured shot-count floor (stitching.min_shot_count) applies on the API
460         # path too ? parity with scripts/run_process_and_stitch.py. Segments WITHOUT
shot_count
461         # metadata are exempt: the floor filters known counts only, so explicitly
462         # requested manual/legacy segments can't silently vanish under the default floor.
463         stitching = getattr(global_config, "stitching", None) or StitchingConfig()
464         kept = []
465         for s in matched_segments:
466             meta = s.get("metadata") or {}
467             sc = meta.get("shot_count") if isinstance(meta, dict) else None
468             try:
469                 if sc is not None and int(sc) < stitching.min_shot_count:
470                     continue
471             except (TypeError, ValueError):
472                 pass  # unparseable count -> treat as unknown, keep
473             kept.append(s)
474         if len(kept) < len(matched_segments):
475             logger.info(f"[compile] stitching.min_shot_count={stitching.min_shot_count}:
dropped "
476                         f"{len(matched_segments) - len(kept)}/{len(matched_segments)}
segments below the floor.")
477         if not kept:
478             raise HTTPException(status_code=400,
479                                 detail=f"All {len(matched_segments)} matching segments fall
below "
480
f"stitching.min_shot_count={stitching.min_shot_count}.")
481
482         # Extract start/end time pairs
483         time_pairs = [(s["start_time"], s["end_time"]) for s in kept]
484
485         # Run compiler with the configured stitching paddings + optional branding watermark
486         output_dir = getattr(getattr(global_config, "output", None), "output_dir", "output")
or "output"
487         compiler = VideoCompiler(output_dir, begin_padding=stitching.begin_padding,
488                                  end_padding=stitching.end_padding,
watermark=stitching.watermark)
489         try:
490             # The stored `filepath` is the SOURCE ref ? a bucket/Drive URI (gs://, s3://,
gdrive://) for a
491             # video ingested via /api/process from a URI, or a plain local path. Resolve it
to a LOCAL file
492             # the stitcher can read: identity for a local path (byte-identical to before),
fetch-to-scratch
493             # for a bucket URI (the same seam the process path uses, _execute_processing
main.py:159).
494             # Without this, compiling a bucket-ingested video 500s ? ffmpeg can't open a
gs:// path.
495             local_src = storage.acquire_local_input(video["filepath"],
```

```
getattr(global_config, "storage", None))
            out_path = compiler.compile_highlights(local_src, time_pairs, out_name)
            # `filepath` is server-local (not reachable by the browser); also return the
ready-to-use
            # download URL so the SPA needn't hand-build it (P6 / PER_RALLY_SELECTION.md
§4.3).
            download_url = f"/api/highlights/{quote(req.video_id)}/{quote(out_name)}"
            return {"status": "success", "filepath": out_path, "download_url": download_url}
        except Exception as e:
            raise HTTPException(status_code=500, detail=f"Highlight stitching failed:
{str(e)}") from e

    # --- CLI Debug Utility & Orchestration ---
    def run_cli_diagnose_clip(video_path: str, timestamp: float):
        """CLI debugging utility to examine segment properties around a timestamp."""
        print("=" * 60)
        print(f"DEBUGGING VIDEO CLIP: {video_path} at {timestamp}s")
        print("=" * 60)

        cfg = load_config()  # legacy wrapper still works, returns dict for now

        # 1. Validation check diagnostics
        print("[DIAGNOSTIC] Running validation framework...")
        results = registry.run_all(video_path, cfg)
        for r in results:
            status_symbol = "[PASS]" if r.passed else "[FAIL]"
            print(f"  {status_symbol} {r.validator_name}: {r.message}")
            if r.details:
                print(f"         Details: {r.details}")

        # 2. Local segment analysis around the timestamp (sampling +/- 5 seconds)
        print("-" * 60)
        print("[DIAGNOSTIC] Analyzing local motion around the timestamp...")
        import cv2
        cap = cv2.VideoCapture(video_path)
        if not cap.isOpened():
            print("[FAIL] OpenCV could not open video.")
            return

        fps = cap.get(cv2.CAP_PROP_FPS)
        total_frames = cap.get(cv2.CAP_PROP_FRAME_COUNT)

        start_frame = max(0, int((timestamp - 3.0) * fps))
        end_frame = min(int(total_frames), int((timestamp + 3.0) * fps))

        # Measure frame-by-frame changes in sample region
        cap.set(cv2.CAP_PROP_POS_FRAMES, start_frame)
        prev_gray = None
        motions = []

        for _ in range(start_frame, end_frame):
            ret, frame = cap.read()
            if not ret:
                break
            gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)
            gray = cv2.GaussianBlur(gray, (21, 21), 0)

            if prev_gray is not None:
                diff = cv2.absdiff(prev_gray, gray)
                _, thresh = cv2.threshold(diff, 25, 255, cv2.THRESH_BINARY)
                motion_ratio = cv2.countNonZero(thresh) / (thresh.shape[0] *
thresh.shape[1])
                motions.append(motion_ratio)
            prev_gray = gray

```

```
556         cap.release()
557
558         if motions:
559             avg_motion = sum(motions) / len(motions)
560             # Support both legacy dict (from wrapper) and future direct model
561             if hasattr(cfg, "indexing"):
562                 threshold = getattr(cfg.indexing, "motion_threshold", 0.08)
563             else:
564                 threshold = cfg.get("indexing", {}).get("motion_threshold", 0.08)  # type:
ignore[attr-defined]
565             print(f"  Sample Frame Count: {len(motions)}")
566             print(f"  Average motion index around timestamp: {round(avg_motion, 4)}
(threshold: {threshold})")
567             if avg_motion > threshold:
568                 print("  Result: [ACTIVE PLAY] High motion detected. Segment classifies as
active play.")
569             else:
570                 print("  Result: [DEAD TIME] Idle movement. Segment classified as dead
time.")
571         else:
572             print("  [ERROR] No visual frames were extracted in the sample range.")
573
574         print("=" * 60)
575
576
577     def _enforce_startup_or_exit(config) -> None:
578         """M5: run the per-profile startup checks for the server/CLI path and
``sys.exit(1)`` on a
579         FATAL incoherent profile (e.g. cloud-serving pointed at local-only storage) BEFORE
any work.
580         A strict no-op when no ``deployment.profile`` is selected (today's behaviour). No
GPU probe
581         here ? the server stays torch-light; the detector healthcheck is the GPU authority.
Extracted
582         so the fail-fast wiring + the default-OFF no-op are unit-testable."""
583         try:
584             enforce_startup_checks(config, logger=logger)
585         except StartupCheckError as e:
586             logger.error("[STARTUP] refusing to start: %s", e)
587             sys.exit(1)
588
589
590     def main():
591         parser = argparse.ArgumentParser(description="Sports Highlight Indexer
Orchestrator")
592         parser.add_argument("--video-path", help="Local path to the MP4 sports video file")
593         parser.add_argument("--output-dir", default="output", help="Directory where
processed clips/highlights are saved")
594         parser.add_argument("--setup", action="store_true", help="Launch dependency checker
and setup utility")
595         parser.add_argument("--debug-clip", type=float, help="Timestamp (in seconds) of a
video clip to run detailed diagnostics on")
596         parser.add_argument("--server", action="store_true", help="Run the FastAPI local API
server")
597         parser.add_argument("--port", type=int, default=8000, help="Port to run the FastAPI
server on")
598         parser.add_argument("--max-frames", type=int, help="Limit processing to the first N
sub-sampled frames for debugging")
599
600         available_segmenters = list(segmenter_registry.list_available().keys())
601         parser.add_argument("--segmenter", choices=available_segmenters, help=f"Choose rally
detection segmenter. Available: {available_segmenters}")
602         parser.add_argument("--trim-start", type=float, help="Start time in seconds for the
debug trim segment feature")
603         parser.add_argument("--trim-end", type=float, help="End time in seconds for the
```

```
debug trim segment feature")
604         parser.add_argument("--trim-output", help="Output filename for the debug trimmed
segment (saves in output-dir unless absolute path)")
605
606         args = parser.parse_args()
607
608         if args.setup:
609             # Invoke the diagnostics/bootstrap script (renamed from the old root
610             # setup.py to scripts/doctor.py so it no longer shadows the PEP 517
611             # build system). Resolve its path from this file so `indexer --setup`
612             # works regardless of the caller's cwd.
613             import subprocess
614             doctor_path = os.path.join(
615                 os.path.dirname(os.path.dirname(os.path.abspath(__file__))),
616                 "scripts", "doctor.py",
617             )
618             print(f"[INFO] Invoking diagnostics: {doctor_path} ...")
619             subprocess.run([sys.executable, doctor_path])
620             return
621
622         if args.trim_start is not None and args.trim_end is not None:
623             if not args.video_path:
624                 print("[ERROR] Please provide --video-path to extract a segment.")
625                 return
626
627             # Determine output path
628             out_filename = args.trim_output or "trimmed_segment.mp4"
629             if os.path.isabs(out_filename):
630                 out_path = out_filename
631                 out_dir = os.path.dirname(out_path)
632                 out_name = os.path.basename(out_path)
633             else:
634                 out_dir = resolve_output_dir(args.output_dir)  # P0a: app-home aware (CWD-
identical when unset)
635                 out_path = os.path.join(out_dir, out_filename)
636                 out_name = out_filename
637
638             os.makedirs(out_dir, exist_ok=True)
639             print(f"[CLI] Extracting debug segment from {args.trim_start}s to
{args.trim_end}s from {args.video_path}...")
640
641             # global_config isn't initialized this early; load the typed config so the trim
642             # clip carries the same configured stitching paddings + watermark as a real
643             # highlight (watermark default-on).
644             stitching = load_app_config().stitching
645             compiler = VideoCompiler(out_dir, begin_padding=stitching.begin_padding,
646                                      end_padding=stitching.end_padding,
watermark=stitching.watermark)
647             try:
648                 res_path = compiler.compile_highlights(args.video_path, [(args.trim_start,
args.trim_end)], out_name)
649                 print(f"[SUCCESS] Trimmed segment saved to: {res_path}")
650             except Exception as e:
651                 print(f"[ERROR] Failed to compile trimmed segment: {e}")
652             return
653
654         if args.debug_clip is not None:
655             if not args.video_path:
656                 print("[ERROR] Please provide --video-path to debug a specific clip.")
657                 return
658             run_cli_diagnose_clip(args.video_path, args.debug_clip)
659             return
660
661         # Initialize Global Config and DB
662         global global_config, db_instance
```

```
663         global_config = load_app_config()  # returns typed AppConfig
664         _enforce_startup_or_exit(global_config)  # M5: fail fast on an incoherent ACTIVE
profile
665
666         # The CLI --output-dir overrides the configured output directory. It now lives
667         # on the typed model (OutputConfig.output_dir), so every consumer ? including
668         # /api/compile ? reads the same value instead of a write-only runtime hack.
669         # P0a: a RELATIVE output dir roots under the app-home (RALLY_APP_HOME); with the env
670         # unset, resolve_output_dir("output") == "output" (CWD-relative) ? byte-identical to
before.
671         global_config.output.output_dir = resolve_output_dir(args.output_dir)
672         os.makedirs(global_config.output.output_dir, exist_ok=True)
673
674         db_path = os.path.join(global_config.output.output_dir,
global_config.output.db_name)
675         db_instance = Database(db_path)
676
677         # Crash recovery (#16): jobs left queued/running by a previous process are dead ?
678         # the executor is in-process. Mark them failed so pollers see a terminal state.
679         swept = db_instance.fail_stale_jobs()
680         if swept:
681             logger.warning(f"[JOBS] Swept {swept} stale job(s) from a previous run ?
failed.")
682
683         # CLI processing mode (no web UI needed)
684         if args.video_path and not args.server:
685             print(f"[CLI] Processing video file: {args.video_path}")
686             # M2: bare/local path = identity (today's behaviour); a bucket/Drive URI is
fetched to scratch.
687             # The local-existence fast-fail only applies to a local input (stat the RESOLVED
key, not the
688             # raw URI); a remote fetch fails loudly. An unsupported scheme is a clean error,
not a crash.
689             storage_cfg = getattr(global_config, "storage", None)
690             try:
691                 input_ref = storage.resolve_input_ref(args.video_path, storage_cfg)
692             except ValueError as e:
693                 print(f"[FAIL] {e}")
694                 return
695             if input_ref.backend == "local" and not os.path.exists(input_ref.key):
696                 print(f"[FAIL] Video file not found: {args.video_path}")
697                 return
698             local_video = storage.acquire_local_input(args.video_path, storage_cfg)
699
700             video_id = compute_video_id(local_video)
701             was_reingest = db_instance.get_video(video_id) is not None
702
703             # Validate (with-context variant surfaces ffprobe_meta for the report)
704             print("[CLI] Validating...")
705             val_results, val_context = registry.run_all_with_context(local_video,
global_config)
706             failures = [r for r in val_results if not r.passed]
707
708             # Save video status
709             status = "failed" if failures else "processed"
710             db_instance.add_video(video_id, args.video_path, status, [r.model_dump() for r
in val_results])
711
712             seg_name = args.segmenter or getattr(getattr(global_config, "indexing", None),
"default_segmenter", "motion")
713             segmenter_actual = None
714             segmentation_ran = False
715             try:
716                 print("\n" + "="*40)
717                 print("VALIDATION SUMMARY")
```

```
                print("="*40)
                for r in val_results:
                    status_symbol = "[PASS]" if r.passed else "[FAIL]"
                    print(f"{status_symbol} {r.validator_name}: {r.message}")
                print("="*40 + "\n")

                if failures:
                    print("[FAIL] Video failed checks. Indexing halted.")
                    return

                # Index
                segmenter_cls = segmenter_registry.get(seg_name)
                if not segmenter_cls:
                    print(f"[FAIL] Segmenter '{seg_name}' not found.")
                    return

                print(f"[CLI] Indexing play segments using segmenter: {seg_name}...")
                segmenter_actual = seg_name
                play_wm, cand_wm = db_instance.segment_watermarks(video_id)
                segmenter = segmenter_cls(db_instance, global_config)
                result = segmenter.process_video(video_id, local_video,
max_frames=args.max_frames)
                segmentation_ran = True

                if isinstance(result, Failure):
                    print(f"[FAIL] Segmenter '{seg_name}' failed: {result.message}")
                    print("[FAIL] Indexing halted for video.")
                    segments = []
                else:
                    segments = result.value if hasattr(result, "value") else result
                    print(f"[SUCCESS] Process complete. Indexed {len(segments)} play
segments inside database: {db_path}")
                # Destroy-AFTER-confirm: keep new on success, restore prior on
empty/failure.
                _finalize_reingest(video_id, segments, play_wm, cand_wm)
            finally:
                # Always emit the per-video observability report. Never raises.
                paths = emit_indexer_report(
                    db=db_instance, video_id=video_id, video_path=args.video_path,
config=global_config,
                    val_results=val_results, val_context=val_context,
                    segmenter_requested=args.segmenter, segmenter_actual=segmenter_actual,
                    was_reingest=was_reingest, segmentation_ran=segmentation_ran,
                )
                if paths:
                    print(f"[REPORT] Technical report : {paths.get('technical_md')}")
                    print(f"[REPORT] Customer summary : {paths.get('customer_md')}")
                    print(f"[REPORT] Machine JSON     : {paths.get('json')}")
            return

        # Start Local Server Mode
        if args.server or not args.video_path:
            print(f"[SERVER] Starting local FastAPI server on
http://localhost:{args.port}...")
            # Resume self-scheduled work orphaned by the restart (EXECUTION_POLICY.md P3):
the
            # in-process wake-timer for any parked 'waiting' job died with the old process,
so
            # kick one drain ? it reclaims due jobs and re-arms the wake-timer for the rest.
            _get_executor().submit(_drain_due_jobs)
            uvicorn.run(app, host="127.0.0.1", port=args.port)

    if __name__ == "__main__":
        # `python -m backend.main` runs THIS file as `__main__`, but `job_worker`/`uploads`
resolve the
```

```
775         # shared mutable state (`db_instance`, `global_config`) via `import backend.main` ?
a SEPARATE
776         # module object. If `main()` runs here it configures `__main__`'s globals, leaving
777         # `backend.main.db_instance` = None; the async-job drain then crashes silently on
778         # `None.claim_due_job()` and every submitted job hangs in 'queued' forever. Dispatch
through
779         # `backend.main` so `main()` configures the module everyone actually reads. (No
recursion:
780         # imported as `backend.main`, `__name__` != "__main__", so this block doesn't re-
fire.)
781         import backend.main as _bm
782
783         _bm.main()
784
```

9. assistant (2026-06-21T14:42:35.877Z)

Now let me look at the diff and the packaging plan to understand what behaviour changed, and check whether the doctor-from-subdir behaviour change is documented.

10. user (2026-06-21T14:42:37.681Z)

```
 .gitattributes                                    |  10 +
 .github/workflows/ci.yml                          |  28 +
 .gitignore                                        | Bin 924 -> 1117 bytes
 CLAUDE.md                                         |  19 +-
 backend/api/__init__.py                           |   1 +
 backend/api/auth.py                               | 127 +++
 backend/api/job_worker.py                         | 184 +++++
 backend/api/uploads.py                            | 155 ++++
 backend/billing.py                                |  60 ++
 backend/compute/__init__.py                       |  61 ++
 backend/compute/base.py                           |  53 ++
 backend/compute/cloud_run.py                      | 155 ++++
 backend/config/__init__.py                        |  26 +-
 backend/config/loader.py                          | 109 ++-
 backend/config/models.py                          | 173 ++++-
 backend/config/presets.py                         | 132 ++++
 backend/config/startup.py                         | 182 +++++
 backend/eval/ablation.py                          | 162 +---
 backend/eval/ablation_pdf.py                      | 141 ++++
 backend/eval/experiment.py                        |  82 ++
 backend/eval/fusion_audio.py                      |  43 +-
 backend/eval/fusion_compare.py                    |  39 +-
 backend/eval/golden_fixtures.py                   | 848 ++++++++++++++++++++++
 backend/eval/golden_real_fixtures.py              | 343 +++++++++
 backend/eval/rally_seg_eval.py                    | 415 +++++++++-
 backend/eval/tolerance_metrics.py                 | 277 +++++++
 backend/eval/windowing.py                         |  41 +-
 backend/infrastructure/database.py                | 717 ++++++++++-------
 backend/main.py                                   | 439 ++++----
 backend/pipeline/detectors/court_gate.py          |  99 +++
 backend/pipeline/detectors/device.py              |  56 ++
 backend/pipeline/detectors/native_wasb_runner.py  | 263 +++++++
 backend/pipeline/detectors/stub_runner.py         | 157 ++++
 backend/pipeline/detectors/tracknet_runner.py     | 196 ++++-
 backend/pipeline/detectors/wasb_infer.py          |  43 +-
 backend/pipeline/segmenters/_keyhint.py           |  13 +
 backend/pipeline/segmenters/fusion_hybrid.py      |  29 +-
 backend/pipeline/segmenters/gemini.py             | 554 ++++++++------
 backend/pipeline/segmenters/motion.py             |  30 +-
 backend/pipeline/segmenters/trajectory_hybrid.py  |  68 +-
 backend/pipeline/segmenters/wasb_hybrid.py        |   8 +
 backend/pipeline/segmenters/yolo_hybrid.py        | 161 ++--
 backend/pipeline/stitcher/compiler.py             | 224 +++---
```

```
backend/providers/gemini.py                          |   4 +
backend/results.py                                   |  20 +-
backend/storage/__init__.py                          | 126 +++
backend/storage/base.py                              |  49 ++
backend/storage/gcs.py                               |  63 ++
backend/storage/gdrive.py                            | 135 ++++
backend/storage/local.py                             |  69 ++
backend/storage/ref.py                               |  98 +++
backend/storage/s3.py                                |  78 ++
backend/utils/paths.py                               |  18 +-
backend/utils/workspace.py                           | 114 +++
backend/worker/__init__.py                           |   7 +
backend/worker/__main__.py                           | 517 ++++++++++++++
deploy/cloudrun/Dockerfile                           |  61 ++
deploy/cloudrun/README.md                            |  61 ++
deploy/digitalocean/README.md                        |  75 ++
deploy/digitalocean/bootstrap.sh                     |  59 ++
deploy/digitalocean/gpu_setup.sh                     |  69 ++
deploy/digitalocean/mount_scratch.sh                 |  46 ++
deploy/gcp/README.md                                 | 257 +++++++
deploy/gcp/create_gpu_vm.sh                          |  58 ++
deploy/gcp/ops-agent-startup.sh                      |  20 +
deploy/gcp/provision.sh                              |  71 ++
deploy/gcp/teardown.sh                               |  47 ++
docs/BACKLOG.md                                      |   2 +-
docs/CLOUD_GPU_DRYRUN_GUIDE.md                       | 202 +++++
docs/CODE_MAP.md                                     |  13 +-
docs/COMMERCIALIZATION.md                            |   2 +-
docs/COMPUTE_DECOUPLED_SERVING/README.md             | 169 ++++
docs/COMPUTE_DECOUPLED_SERVING/TESTING_STRATEGY.md   |  83 ++
docs/COMPUTE_DECOUPLED_SERVING/architecture.svg      | 130 ++++
docs/COMPUTE_DECOUPLED_SERVING/runlogs/README.md     |  35 +
.../runlogs/RUNLOG_truly-local_TEMPLATE.md           |  49 ++
docs/DESKTOP_APP_PLAN.md                             | 164 ++++
docs/DOC_STATUS.md                                   |  92 +++
docs/EXPERIMENT_HARNESS.md                           |   2 +-
docs/GOLDEN_REGRESSION_FIXTURES.md                   | 216 ++++++
docs/HOSTING_PLAN.md                                 |   4 +
docs/HOW_RALLY_DETECTION_WORKS.md                    |   2 +-
docs/I18N_PLAN.md                                    |   9 +-
docs/MULTI_SIGNAL_FUSION_PLAN.md                     |  24 +-
docs/NEXT_STEPS.md                                   |  57 +-
docs/PER_VIDEO_WORKSPACE.md                          | 112 +++
docs/PLATFORM_ARCHITECTURE.md                        |   4 +-
docs/PROJECT_DOC_TEMPLATE.md                         |  74 ++
docs/PROMINENT_COURT_DETECTION.md                    | 126 +++
docs/QUALITY_ITERATIONS.md                           | 378 ++++++++-
docs/R1_MILESTONE_LAUNCH_REPORT.md                   |  89 +++
docs/R2_EVAL_METRIC_DESIGN.md                        | 195 +++++
docs/RALLY_DETECTION_GAP_CLOSURE.md                  | 104 +++
docs/RALLY_DETECTION_QUALITY_REPORT.md               | 478 ++++++++++++
docs/RALLY_QUALITY_RESEARCH.md                       |  70 +-
docs/README.md                                       |  27 +-
docs/REAL_FOOTAGE_VALIDATION_REPORT.md               | 119 +++
docs/ROADMAP.md                                      |   4 +-
docs/SETUP_NEW_MACHINE.md                            | 312 ++++++++
docs/UI_PROPOSAL.md                                  |   6 +
...R_VIDEO_OUTPUT_LAYOUT-SUPERSEDED-2026-06-21.md} |  12 +-
docs/archives/decisions/DECISIONS.md                 | 205 +++++
.../field-pmf}/FIELD_VISIT_ANSWER_SHEET.md           |   0
.../field-pmf}/FIELD_VISIT_PLAYBOOK.md               |   0
docs/{ => archives/field-pmf}/PMF_FIELD_SIGNALS.md   |   0
.../field-pmf}/PMF_MARKET_RESEARCH.md                |   0
.../field-pmf}/PRODUCT_FIELD_ASSOCIATE_FLYER.md      |   0
docs/archives/field-pmf/README.md                    |  15 +
```

```
.../DECOUPLED_COMPUTE/01-EXISTING-PLANS-AND-GAP.md | 137 ++++
.../02-COMPUTE-MAP-AND-CONTRACT.md                 |  78 ++
.../DECOUPLED_COMPUTE/03-PLATFORM-OPTIONS.md       |  94 +++
.../DECOUPLED_COMPUTE/04-BUILD-AND-TEST-PLAN.md    | 161 ++++
.../DECOUPLED_COMPUTE/05-COST-ATTRIBUTION.md       | 105 +++
.../06-RISKS-AND-OPEN-QUESTIONS.md                 |  85 +++
.../DECOUPLED_COMPUTE/07-COMPUTE-ABSTRACTION.md    | 168 ++++
.../DEPLOY_REPORT_GCP_2026-06-20.md                | 110 +++
.../past_projects/DECOUPLED_COMPUTE/HANDOFF.md     | 108 +++
.../past_projects/DECOUPLED_COMPUTE/README.md      | 126 +++
docs/archives/past_projects/README.md              |   3 +-
.../2026-06-18-lovo-resweep-n13/README.md          |  75 ++
.../2026-06-18-lovo-resweep-n13/lovo_resweep.py    | 111 +++
docs/archives/quality-iterations/README.md         |   1 +
.../roora-vendor}/GOLDEN_SET_VENDORING.md          |   0
docs/archives/roora-vendor/README.md               |  14 +
.../roora-vendor}/ROORA_LABELING_GUIDELINES.md     |   0
docs/{ => data_pipeline}/GOLDEN_DATA_SHARING.md    |   2 +-
.../GOLDEN_SET_IMPLEMENTATION.md                   |   2 +-
.../GOLDEN_SET_PHASE1_VIDEOS.md                    |   2 +-
docs/data_pipeline/GOLDEN_VIDEOS.md                |  60 ++
docs/data_pipeline/README.md                       |  16 +
.../VIDEO_RECORDING_GUIDELINES.md                  |  21 +
eval_baselines/fixtures/README.md                  |  70 ++
.../fixtures/clean_single_rally/expected.json      |  39 +
.../fixtures/clean_single_rally/labels.csv         |   2 +
.../fixtures/clean_single_rally/provenance.json    |  23 +
.../fixtures/clean_single_rally/trajectory.csv     | 121 +++
eval_baselines/fixtures/manifest.json              |  62 ++
.../multi_rally_with_deadtime/expected.json        |  54 ++
.../fixtures/multi_rally_with_deadtime/labels.csv  |   3 +
.../multi_rally_with_deadtime/provenance.json      |  23 +
.../multi_rally_with_deadtime/trajectory.csv       | 331 ++++++++
.../fixtures/occlusion_break/expected.json         |  54 ++
eval_baselines/fixtures/occlusion_break/labels.csv |   3 +
.../fixtures/occlusion_break/provenance.json       |  23 +
.../fixtures/occlusion_break/trajectory.csv        | 331 ++++++++
eval_baselines/nightly_baseline.json               | 569 ++++++++++++++
mypy.ini                                           |  13 +
pyproject.toml                                     |  13 +
scripts/doctor.py                                  |  18 +-
scripts/nightly_regression.ps1                     |  83 ++
scripts/nightly_regression.py                      | 633 ++++++++++++++++
scripts/register_nightly_task.ps1                  |  70 ++
scripts/run_phase3_eval.py                         |  11 +-
scripts/run_process_and_stitch.py                  |  23 +-
tests/test_ablation.py                             |  47 +-
tests/test_api_endpoints.py                        | 309 +++++++-
tests/test_async_jobs.py                           |   8 +-
tests/test_audio_features.py                       |  26 +
tests/test_billing.py                              | 192 +++++
tests/test_compute_backend.py                      | 289 +++++++
tests/test_config.py                               |  20 +
tests/test_config_deployment.py                    | 371 +++++++++
tests/test_config_wasb.py                          |  16 +-
tests/test_court_gate.py                           | 103 +++
tests/test_database.py                             | 170 ++++-
tests/test_device_context.py                       |  47 ++
tests/test_edge_auth.py                            | 222 ++++++
tests/test_experiment.py                           |  72 ++
tests/test_fusion_compare.py                       |  25 +
tests/test_fusion_hybrid.py                        |  10 +
tests/test_generate_highlights.py                  | 174 ++++-
tests/test_golden_fixtures.py                      | 138 ++++
tests/test_golden_fixtures_split.py                |  87 +++
```

```
 tests/test_golden_real_fixtures.py                | 504 ++++++++++++
 tests/test_input_source.py                        | 152 ++++
 tests/test_job_worker_loop.py                     |   8 +-
 tests/test_job_worker_module_seam.py              |  97 +++
 tests/test_motion_segmenter.py                    | 222 ++++++
 tests/test_native_wasb_runner.py                  | 383 ++++++++++
 tests/test_nightly_regression.py                  | 343 +++++++++
 tests/test_output_base.py                         |  90 +++
 tests/test_profile_parity.py                      | 140 ++++
 tests/test_provider_gemini.py                     |  21 +
 tests/test_rally_seg_eval.py                      | 155 +++-
 tests/test_rally_seg_eval_report.py               | 306 ++++++++
 tests/test_results.py                             |  50 ++
 tests/test_served_gate_a_regression.py            |  96 ++-
 tests/test_server_entrypoint.py                   |  95 +++
 tests/test_startup_checks.py                      | 226 ++++++
 tests/test_storage.py                             | 328 ++++++++
 tests/test_stub_runner.py                         | 129 ++++
 tests/test_tolerance_metrics.py                   | 188 +++++
 tests/test_tracknet_runner.py                     |  96 +++
 tests/test_training_gen0.py                       |  56 ++
 tests/test_upload_download.py                     |   3 +-
 tests/test_wasb_cache.py                          |  20 +
 tests/test_windowing.py                           |  37 +-
 tests/test_worker.py                              | 309 ++++++++
 tests/test_workspace.py                           | 122 +++
 tests/test_yolo_hybrid.py                         |  41 +
 tools/generate_highlights.py                      | 118 ++-
 training/gen0/harness.py                          |  50 +-
 202 files changed, 22017 insertions(+), 1375 deletions(-)
```

11. user (2026-06-21T14:42:38.742Z)

```
diff --git a/backend/config/loader.py b/backend/config/loader.py
index 463d9ff..dd34507 100644
--- a/backend/config/loader.py
+++ b/backend/config/loader.py
@@ -10,18 +10,67 @@ from __future__ import annotations

 import json
 import logging
+import os
 from pathlib import Path
 from typing import Any

 from pydantic import BaseModel

 from .models import AppConfig
+from .presets import (
+    COMPUTE_TARGET_FOR_PROFILE,
+    PROFILE_PRESETS,
+    is_known_profile,
+    preset_for,
+    suggest_profile,
+)

 logger = logging.getLogger(__name__)

 DEFAULT_CONFIG_PATH = Path("config.json")


+def _deep_merge(base: dict[str, Any], override: dict[str, Any]) -> dict[str, Any]:
+    """Recursively overlay ``override`` onto ``base`` (override wins; dicts merge, scalars
+    replace). Used ONLY for the profile-preset precedence path so a config.json sub-key
+    overrides a preset sub-key while sibling preset/default sub-keys survive ? the no-profile
```

```
+    path keeps the original shallow ``{**defaults, **file_data}`` merge unchanged."""
+    out = dict(base)
+    for key, val in override.items():
+        if isinstance(val, dict) and isinstance(out.get(key), dict):
+            out[key] = _deep_merge(out[key], val)
+        else:
+            out[key] = val
+    return out
+
+
+def _resolve_explicit_profile(file_data: dict[str, Any]) -> str:
+    """The EXPLICITLY chosen deployment profile, env winning over config.json. ``""`` if none
+    is set. Auto-detect is deliberately NOT consulted here ? it only suggests (D15), so a bare
+    environment never silently selects a profile (and never silently spends on cloud).
+
+    An *unrecognised* ``DEPLOYMENT_PROFILE`` env value warns and FALLS THROUGH to the file's
+    profile (rather than suppressing it) ? so an env typo can't leave the recorded profile
+    asserting a preset that was never applied. A bad value in config.json is left to surface as
+    a hard, typed-Literal error downstream (config-file correctness)."""
+    env_profile = os.environ.get("DEPLOYMENT_PROFILE")
+    if env_profile and env_profile.strip():
+        ep = env_profile.strip()
+        if is_known_profile(ep):
+            return ep
+        logger.warning(
+            "[CONFIG] unrecognized DEPLOYMENT_PROFILE env value %r (valid: %s); ignoring it "
+            "and falling back to config.json deployment.profile.", ep, ",
".join(PROFILE_PRESETS),
+        )
+        # fall through to the file's profile (env typo must not suppress a valid file choice)
+    dep = file_data.get("deployment")
+    if isinstance(dep, dict):
+        prof = dep.get("profile")
+        if isinstance(prof, str) and prof.strip():
+            return prof.strip()
+    return ""
+
+
 def _unknown_key_paths(data: dict[str, Any], model_cls: type[BaseModel], prefix: str = "") ->
list[str]:
     """Dotted paths in `data` that the typed config will not honor.

@@ -72,6 +121,15 @@ def load_config(path: str | Path | None = None) -> AppConfig:
             # In production we might log here.
             logger.warning(f"Failed to read {cfg_path}: {exc}. Using defaults + partial data.")

+    # A config.json that is valid JSON but not an OBJECT (e.g. `[]`, `42`, `"x"`, `null`) would
+    # otherwise crash startup: a truthy non-mapping hits `.items()` in _unknown_key_paths, and
any
+    # non-mapping breaks the `{**defaults, **file_data}` unpack. Degrade to defaults + warn,
per the
+    # loader's "bad input is non-fatal" contract.
+    if not isinstance(file_data, dict):
+        logger.warning("%s is not a JSON object (got %s); ignoring it and using defaults.",
+                       cfg_path, type(file_data).__name__)
+        file_data = {}
+
     if file_data:
         unknown = _unknown_key_paths(file_data, AppConfig)
         if unknown:
@@ -82,8 +140,55 @@ def load_config(path: str | Path | None = None) -> AppConfig:
                 cfg_path, ", ".join(sorted(unknown)),
             )

-    # Merge: start from defaults, overlay file data (Pydantic will validate/coerce).
```

```
-    merged = {**_get_builtin_defaults(), **file_data}
+    # Precedence: built-in defaults < profile preset < config.json (< env/--set, applied at
+    # consumption sites downstream). A *deployment profile* is opt-in ? it is applied ONLY when
+    # explicitly selected (DEPLOYMENT_PROFILE env or config.json deployment.profile). With no
+    # profile the merge is byte-identical to the pre-M1 loader, so the default path is
unchanged.
+    defaults = _get_builtin_defaults()
+    profile = _resolve_explicit_profile(file_data)
+
+    if profile and not is_known_profile(profile):
+        # Only an unrecognised value from config.json reaches here (env typos already fell back
+        # in _resolve_explicit_profile). Warn + apply no preset; the bad value also hits the
+        # typed Literal at model_validate below, a hard clear error ? loud, never silent.
+        logger.warning(
+            "[CONFIG] unrecognized deployment profile %r (valid: %s); applying no preset.",
+            profile, ", ".join(PROFILE_PRESETS),
+        )
+        profile = ""
+
+    if profile:
+        merged = _deep_merge(_deep_merge(defaults, preset_for(profile)), file_data)
+        # Record the profile actually applied ? the env may have chosen it over config.json,
+        # so re-stamp it onto the merged deployment section to keep the record honest.
+        merged.setdefault("deployment", {})
+        if isinstance(merged["deployment"], dict):
+            merged["deployment"]["profile"] = profile
+            ct = merged["deployment"].get("compute_target")
+            canonical = COMPUTE_TARGET_FOR_PROFILE.get(profile)
+            if not ct and canonical:
+                # An active profile with no explicit compute_target (e.g. config.json blanked
it
+                # to "") gets its canonical target back ? the record never silently lies about
an
+                # active profile's compute_target (it stays meaningful for the M2+
ComputeBackend).
+                merged["deployment"]["compute_target"] = canonical
+            elif ct and canonical and ct != canonical:
+                logger.warning(
+                    "[CONFIG] deployment profile %r normally uses compute_target=%r but "
+                    "config.json overrides it to %r ? honoring the explicit override.",
+                    profile, canonical, ct,
+                )
+            logger.info("[CONFIG] deployment profile ACTIVE: %s (compute_target=%s)",
+                        profile, merged["deployment"].get("compute_target"))
+    else:
+        # No profile ? the original shallow merge (Pydantic re-defaults nested sections).
+        merged = {**defaults, **file_data}
+        suggestion = suggest_profile()  # cheap, env-only (no torch); SUGGESTS only (D15)
+        if suggestion:
+            logger.debug(
+                "[CONFIG] no deployment.profile set (manual knobs). Environment suggests %r ? "
+                "set deployment.profile or DEPLOYMENT_PROFILE to apply (auto-detect never "
+                "auto-applies ? D15).", suggestion,
+            )

     return AppConfig.model_validate(merged)

diff --git a/backend/main.py b/backend/main.py
index 0f48c7e..6768d30 100644
--- a/backend/main.py
+++ b/backend/main.py
@@ -2,10 +2,8 @@ import os
 import sys
 import argparse
 import logging
```

```
-import threading
-import traceback
 import uuid
-from concurrent.futures import ThreadPoolExecutor
+from urllib.parse import quote
 import uvicorn
 from dotenv import load_dotenv

@@ -31,18 +29,39 @@ from backend.pipeline.segmenters.base import segmenter_registry
 from backend.pipeline.stitcher.compiler import VideoCompiler
 from backend.utils.hashing import compute_video_id  # canonical file-identity hashing
 from backend.utils.paths import resolve_under_root, safe_output_filename, validate_input_path
-from backend.config import load_config as load_app_config, AppConfig, StitchingConfig  # new
typed config
+from backend.config import (  # new typed config
+    load_config as load_app_config,
+    AppConfig,
+    StitchingConfig,
+    enforce_startup_checks,
+    StartupCheckError,
+)
 from backend.results import Failure  # standardized error/result contract
 from backend.reporting import emit_indexer_report
+from backend import storage  # M2: URI-aware input acquisition (local = identity passthrough)
+from backend import billing  # M8: per-job cost ledger (USD internal / INR display)
+from backend.api import auth as edge_auth  # M9: Cloudflare Access edge-auth (default-OFF)

 app = FastAPI(title="Sports Highlight Indexer API")

-# Enable CORS for the local web interface. allow_origins='*' with allow_credentials=True is
-# rejected by browsers per the fetch spec and is an unsafe default to carry into the
-# auth/tenancy work; this tool uses no cookies, so credentials stay off.
+
+def _cors_origins_from_env(env: Optional[Dict[str, str]] = None) -> List[str]:
+    """M9: CORS allow-origins from the ``RALLY_CORS_ALLOW_ORIGINS`` env var (comma-separated),
+    default ``["*"]``. Read from the ENV (not a cwd ``config.json``) so importing
``backend.main``
+    does zero filesystem I/O ? a hosted/multi-tenant deploy sets this to lock CORS to its
origin."""
+    src = os.environ if env is None else env
+    raw = (src.get("RALLY_CORS_ALLOW_ORIGINS", "") or "").strip()
+    origins = [o.strip() for o in raw.split(",") if o.strip()]
+    return origins or ["*"]
+
+
+# CORS for the web UI. allow_origins='*' with allow_credentials=True is rejected by browsers
per the
+# fetch spec + is unsafe to carry into multi-tenant; this tool uses no cookies, so credentials
stay
+# off. The origin list is configurable via the env var above (default ["*"]) so a hosted deploy
can
+# lock it to its origin; the middleware is fixed at startup.
 app.add_middleware(
     CORSMiddleware,
-    allow_origins=["*"],
+    allow_origins=_cors_origins_from_env(),
     allow_credentials=False,
     allow_methods=["*"],
     allow_headers=["*"],
@@ -64,18 +83,24 @@ def serve_index():
 db_instance: Optional[Database] = None
 global_config: Optional[AppConfig] = None  # now typed (Pydantic model)

-# Async job executor (DEFERRED_HARDENING_PLAN #16). max_workers=1 on purpose: a single
-# self-hosted box serializes GPU work, and the Gemini provider is rate-fragile under
```

```
-# parallel uploads (2026-06-10 ops learning: serial uploads + backoff). The segmenters
-# already parallelize their AI handoff internally via their own pools.
-_job_executor: Optional[ThreadPoolExecutor] = None
-
-def _get_executor() -> ThreadPoolExecutor:
-    """Lazy module-global executor; tests monkeypatch this for inline execution."""
-    global _job_executor
-    if _job_executor is None:
-        _job_executor = ThreadPoolExecutor(max_workers=1, thread_name_prefix="jobworker")
-    return _job_executor
+# Async job worker ? drain/wake loop (DEFERRED_HARDENING_PLAN #16, EXECUTION_POLICY.md P3)
+# now lives in backend.api.job_worker. Re-exported here so the names stay addressable on
+# `backend.main` (`JobWaiting`, `_drain_due_jobs`, `_arm_wake_timer`, `_get_executor`,
+# `_run_claimed_job`, `_MAX_DRAIN_WAKE_SEC`, ...). The worker resolves these seams THROUGH
+# this module object at call-time, so `monkeypatch.setattr(backend.main, ...)` still
+# intercepts the inter-function calls (a parked job arms the PATCHED `_arm_wake_timer`,
+# so no real daemon timer leaks in tests). Behavior is unchanged by the extraction.
+from backend.api.job_worker import (  # noqa: F401  (re-exports: names stay addressable on
backend.main)
+    JobWaiting,
+    _arm_wake_timer,
+    _drain_due_jobs,
+    _get_executor,
+    _job_to_request,
+    _MAX_DRAIN_WAKE_SEC,
+    _maybe_record_job_cost,
+    _run_claimed_job,
+    _schedule_next_drain_if_waiting,
+)

 # Legacy thin wrapper for any remaining direct calls during transition.
 # New code should import load_app_config / AppConfig from backend.config directly.
@@ -120,6 +145,19 @@ def _finalize_reingest(video_id: str, segments, play_wm: int, cand_wm: int)
-> N
 def get_config():
     return global_config

+@app.get("/api/health")
+def health():
+    """Cheap liveness/readiness probe (no auth) for the SPA's offline banner
+    (HOSTING_PLAN.md:42). Reports the configured sport + default segmenter so the UI can
+    confirm the engine is reachable and which detector is wired. Never raises."""
+    sport = getattr(global_config, "sport", None)
+    indexing = getattr(global_config, "indexing", None)
+    return {
+        "status": "ok",
+        "sport": sport,
+        "default_segmenter": getattr(indexing, "default_segmenter", None),
+    }
+
 def _execute_processing(req: ProcessRequest, progress=None) -> Dict[str, Any]:
     """The full hash ? validate ? segment ? report pipeline for one video.

@@ -135,13 +173,18 @@ def _execute_processing(req: ProcessRequest, progress=None) -> Dict[str,
Any]:
     notify = progress or (lambda stage: None)

     notify("hashing")
-    video_id = compute_video_id(req.video_path)
+    # M2: resolve the input to a LOCAL path. A bare/local path is returned unchanged (identity
?
+    # byte-identical to today); a bucket/Drive URI is fetched to scratch first. The original
+    # req.video_path (the source URI) is kept for the DB record + report; only the file-reading
+    # consumers (hash / validators / segmenter) use the resolved local path.
+    local_video = storage.acquire_local_input(req.video_path, getattr(global_config, "storage",
```

```
None))
+    video_id = compute_video_id(local_video)
     was_reingest = db_instance.get_video(video_id) is not None

     # 1. Run pluggable validators (with-context variant surfaces ffprobe_meta for the report)
     notify("validating")
     logger.info(f"Running validations for {req.video_path}...")
-    val_results, val_context = registry.run_all_with_context(req.video_path, global_config)
+    val_results, val_context = registry.run_all_with_context(local_video, global_config)
     failures = [r for r in val_results if not r.passed]

     # typed AppConfig: attribute access for the default segmenter (with safe fallback)
@@ -189,7 +232,7 @@ def _execute_processing(req: ProcessRequest, progress=None) -> Dict[str,
Any]:
         play_wm, cand_wm = db_instance.segment_watermarks(video_id)
         segmenter = segmenter_cls(db_instance, global_config)
         try:
-            result = segmenter.process_video(video_id, req.video_path, req.player_pool,
req.max_frames)
+            result = segmenter.process_video(video_id, local_video, req.player_pool,
req.max_frames)
         except Exception:
             # A raising segmenter must not leave half-written rows: restore the pre-run
             # state (same destroy-after-confirm contract as the Failure branch), then let
@@ -235,116 +278,8 @@ def _execute_processing(req: ProcessRequest, progress=None) -> Dict[str,
Any]:
             response["reports"] = paths


-class JobWaiting(Exception):
-    """Raised inside the pipeline to SELF-SCHEDULE a job (EXECUTION_POLICY.md P3,
-    WAIT_AND_RESUME). It means "I can't finish now ? park me and resume in `delay_sec`",
-    NOT a failure. The worker persists `next_poll_at` + `progress` (via `set_job_waiting`),
-    releases the thread, and re-claims the job when due (`claim_due_job`). No work to date
-    is the segmenter's `_execute_processing`, run UNDER the destroy-after-confirm contract.
-
-    The wiring is additive: the production segmenter path never raises this today (zero
-    behaviour change), so a job runs exactly as before. The hook is what a later slice
-    (resumable AI handoff) raises once a long op exhausts its in-process wait budget.
-    """
-
-    def __init__(self, delay_sec: float, progress: Optional[str] = None) -> None:
-        super().__init__(f"job parked for {delay_sec:.0f}s")
-        self.delay_sec = max(0.0, float(delay_sec))
-        self.progress = progress
-
-
-def _job_to_request(job: Dict[str, Any]) -> ProcessRequest:
-    """Reconstruct the submit-time ProcessRequest from a claimed job row, so a job
-    re-claimed by `claim_due_job` (queued OR resumed-from-waiting) runs identically to
-    the original submit. The row stores exactly the ProcessRequest fields."""
-    return ProcessRequest(
-        video_path=job["video_path"],
-        player_pool=job.get("player_pool"),
-        max_frames=job.get("max_frames"),
-        segmenter=job.get("segmenter"),
-    )
-
-
-def _run_claimed_job(job: Dict[str, Any]) -> None:
-    """Run ONE job already claimed (`status='running'`) by `claim_due_job`, recording its
-    terminal state ? OR parking it for resume on `JobWaiting` (WAIT_AND_RESUME).

-    Job `status` is the EXECUTION state (queued|waiting|running|done|failed): 'done' means
-    the worker finished ? the pipeline's own verdict (validation failure etc.) lives in
```

```
-    result["status"]; 'failed' means the worker itself crashed/raised; 'waiting' means the
-    job self-scheduled and will be re-claimed when `next_poll_at` is due.
-    """
-    job_id = job["id"]
-    req = _job_to_request(job)
-    try:
-        result = _execute_processing(
-            req, progress=lambda stage: db_instance.set_job_progress(job_id, stage))
-        if result.get("video_id"):
-            db_instance.set_job_video_id(job_id, result["video_id"])
-        db_instance.mark_job_done(job_id, result)
-    except JobWaiting as w:
-        # Cooperative re-enqueue: persist progress + next_poll_at, release the worker.
-        logger.info(f"Job {job_id} parked for {w.delay_sec:.0f}s (resume when due).")
-        db_instance.set_job_waiting(job_id, w.delay_sec, progress=w.progress)
-    except Exception as e:
-        logger.error(f"Job {job_id} crashed: {e}\n{traceback.format_exc()}")
-        db_instance.mark_job_failed(job_id, f"{type(e).__name__}: {e}")
-
-
-def _drain_due_jobs() -> int:
-    """One worker tick: claim and run every currently-runnable job (queued, or waiting whose
-    `next_poll_at` is due), earliest-due first, until the table has nothing runnable.
-
-    Each parked job (`JobWaiting` ? status 'waiting') drops out of this drain and is picked
-    up by a FUTURE drain once due; we schedule that follow-up drain via `_schedule_drain` so
-    a single-worker box keeps making progress with no central timer (the DB is the clock).
-    Returns the number of jobs that reached a terminal/parked state this tick.
-    """
-    ran = 0
-    while True:
-        job = db_instance.claim_due_job()
-        if job is None:
-            break
-        ran += 1
-        _run_claimed_job(job)
-    # If anything is parked-but-not-yet-due, arrange a wake-up so it resumes without a new
-    # submit. On the single-box executor this is a delayed re-drain; multi-worker just relies
-    # on the next submit/drain. Best-effort ? never raises into the worker.
-    _schedule_next_drain_if_waiting()
-    return ran
-
-
-def _schedule_next_drain_if_waiting() -> None:
-    """If any job is parked in 'waiting', arm a re-drain for when the soonest one is due, so a
-    self-scheduled job resumes with no further client submit. The drain itself re-checks
-    due-ness, so an early wake is harmless and a clamped wake for a far-future job is fine."""
-    try:
-        delay = db_instance.seconds_until_next_due()
-    except Exception as e:  # pragma: no cover - defensive; never wedge the worker
-        logger.error(f"Could not compute next-due delay: {e}")
-        return
-    if delay is None:
-        return  # nothing waiting
-    # Clamp so a far-future job doesn't pin a thread forever; the drain re-checks due-ness.
-    _arm_wake_timer(max(0.0, min(delay, _MAX_DRAIN_WAKE_SEC)))
-
-
-def _arm_wake_timer(delay_sec: float) -> None:
-    """Arm a single daemon timer that re-submits a drain after ``delay_sec`` (no central
-    scheduler ? the DB is the clock). Isolated as the sole timer-creating seam so tests can
-    monkeypatch it to a no-op and assert the *scheduling decision* without real waiting."""
-    timer = threading.Timer(delay_sec, lambda: _get_executor().submit(_drain_due_jobs))
-    timer.daemon = True
-    timer.start()
```

-

12. assistant (2026-06-21T14:42:41.808Z)

Now let me look at the doctor.py diff specifically and check what tests exist for doctor.py and main.py server entrypoint, plus the packaging plan documentation.

13. user (2026-06-21T14:42:42.998Z)

```
diff --git a/scripts/doctor.py b/scripts/doctor.py
index 5c30f61..b14bda8 100644
--- a/scripts/doctor.py
+++ b/scripts/doctor.py
@@ -22,9 +22,13 @@ def run_pip_install():
     requirements_path = os.path.join(REPO_ROOT, "requirements.txt")
     pip_cmd = [sys.executable, "-m", "pip", "install", "--user", "-r", requirements_path]

+    import platform
     if has_gpu:
         print("[INFO] NVIDIA GPU detected. Configuring PyTorch to download CUDA-enabled
wheels.")
         pip_cmd.extend(["--extra-index-url", "https://download.pytorch.org/whl/cu124"])
+    elif platform.system() == "Darwin":
+        # macOS has no CUDA; the default PyPI wheel already ships CPU + Apple MPS.
+        print("[INFO] macOS detected. Using default PyTorch wheels (CPU + Apple MPS).")
     else:
         print("[INFO] No NVIDIA GPU detected. Defaulting to PyTorch CPU wheels.")
         pip_cmd.extend(["--extra-index-url", "https://download.pytorch.org/whl/cpu"])
@@ -91,13 +95,17 @@ def run_diagnostics():
         print("[FAIL] FFprobe is NOT found in your PATH.")

     if not (ffmpeg_ok and ffprobe_ok):
+        import platform
         print("\n" + "!" * 50)
         print("CRITICAL: FFmpeg and FFprobe are REQUIRED to run this application.")
-        print("How to install FFmpeg on Windows:")
-        print("  Option 1: Open PowerShell as Administrator and run:")
-        print("            winget install Gyan.FFmpeg")
-        print("  Option 2: Download the build from https://ffmpeg.org/download.html")
-        print("            and add its 'bin' directory to your System PATH environment
variable.")
+        _sys = platform.system()
+        if _sys == "Windows":
+            print("Install on Windows:  winget install Gyan.FFmpeg")
+        elif _sys == "Darwin":
+            print("Install on macOS:    brew install ffmpeg")
+        else:
+            print("Install on Linux:    sudo apt-get install -y ffmpeg   (or your distro's
package manager)")
+        print("Or download a build from https://ffmpeg.org/download.html and put ffmpeg/ffprobe
on PATH.")
         print("!" * 50 + "\n")
     else:
         print("[SUCCESS] All essential media command line tools are ready.")
```

14. user (2026-06-21T14:42:44.047Z)

```
7835f34 feat(gcp): always-install Ops Agent on GPU VMs + a cloud-GPU dry-run guide (#291)
aac7633 feat(serving): M9-auth ? Cloudflare Access edge-auth + per-tenant owner_id (default-OFF)
(#290)
e68b583 fix(api): resolve a bucket-ingested video's source to a local file before stitching
(#289)
43eccc1 feat(serving): M8 ? per-job cost ledger + versioned pricebook + /cost API (default-OFF)
(#288)
```

```
8cdf65c feat(serving): M6 ? Cloud Run GPU dispatch shim + worker image (default-OFF, mocked)
(#287)
b5af66e feat(serving): M5 ? truly-local profile per-profile startup checks + L4 profile-parity
test (default-OFF) (#286)
fefa412 docs(serving): resolve §8 design picks (D16/D17) + reconcile per-video P3 vs M3 (#285)
06ff4b7 test(api): lock gs://|gcs:// /api/process -> queryable API-DB video_id (B-P1 guard)
(#284)
cee4baa feat(serving): M4 ? DeviceContext + native WASB CPU-fallback (device="auto", cuda stays
strict) (#283)
2afd54f feat(serving): M3 ? configurable output/scratch base (de-hardcode output/, default-OFF)
(#282)
41d45fd fix(server): drain async jobs under `python -m backend.main` (double-module hang) (#281)
e111add feat(serving): M2 ? URI-aware input (input_backend/input_root, local=identity) (#280)
ce22791 feat(serving): M1 ? deployment.profile + compute_target config (pure, additive, default-
OFF) (#279)
f518e97 docs(serving): M0 APPROVED + lock D13-D15 (switchable profiles, Drive Northstar,
consent-not-surprise) (#278)
49c082a docs(serving): lock the 5 M0 gating decisions (D7-D12) + add the data-flywheel harness
(M11) (#277)
f592281 docs: point I18N_PLAN + UI_PROPOSAL (+ DOC_STATUS) at the khelsutra UX/l10n project
(#275)
f6f15ca docs: isolate field-ops/PMF + Roora into archives/; carve a no-ML docs/data_pipeline/
(#276)
03fa951 docs: resolve DOC_STATUS findings #1 (per-video supersede) + #2 (NEXT_STEPS refresh)
(#274)
aa5e9db docs: add DOC_STATUS register + archival due-diligence convention (#273)
bd42250 docs: spin up COMPUTE_DECOUPLED_SERVING subproject + ADR-014 (#272)
4957b72 fix(worker): configure logging + explain a 0-segment infer (the cold-start blind spot)
(#271)
4134721 docs: archive DECOUPLED_COMPUTE ? DELIVERED (M0-M2 + M3.5 on GCP) (#270)
c8e60ee fix(gen0): provenance-stamp the manifest git_sha off a git-archive tarball (#269)
e6110ce feat(api): GET /api/health + return download_url from /api/compile (M1/P6) (#268)
a555887 docs(deploy): M2 FINALIZED ? Gen-0 weights/0.1.0-l4 pushed; both VMs deleted ($0) (#267)
b05401b docs: drop DigitalOcean ? GCP is the sole compute stack (ADR-013) (#266)
4d546ec fix(gcp): teardown defaults to DELETE (a stopped VM still bills its disk) (#265)
dd30e9b feat(workspace): per-video workspace convention ? design doc + Phase 0 helper (#264)
dc9111b docs(adr): ADR-012 ? GPU-native compute; TPU deferred; Cloud Run+GPU is the serving
model (#263)
d8c5aa0 docs(deploy): report polish ? highlight reel smoke-test result + VM-deleted final state
(#262)
```

15. assistant (2026-06-21T14:42:47.495Z)

The diff stat is against `master` which is far behind. The actual P0a changes are only in the worktree (uncommitted or in a feature branch). Let me check the actual git state of the worktree to find the real P0a diff, and find the test files for doctor and server entrypoint.

16. user (2026-06-21T14:42:49.274Z)

```
On branch feat/packaging-p0a-app-home
Your branch is behind 'origin/master' by 1 commit, and can be fast-forwarded.
  (use "git pull" to update your local branch)

Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
        modified:   backend/config/loader.py
        modified:   backend/main.py
        modified:   scripts/doctor.py

Untracked files:
  (use "git add <file>..." to include in what will be committed)
        backend/utils/app_home.py
        tests/test_app_home.py
```

```
no changes added to commit (use "git add" and/or "git commit -a")
=== branch ===
feat/packaging-p0a-app-home
=== HEAD ===
7835f34 feat(gcp): always-install Ops Agent on GPU VMs + a cloud-GPU dry-run guide (#291)
aac7633 feat(serving): M9-auth ? Cloudflare Access edge-auth + per-tenant owner_id (default-OFF)
(#290)
e68b583 fix(api): resolve a bucket-ingested video's source to a local file before stitching
(#289)
```

17. user (2026-06-21T14:42:49.596Z)

```
tests\test_server_entrypoint.py
```

18. user (2026-06-21T14:42:49.887Z)

```
No files found
```

19. assistant (2026-06-21T14:42:52.850Z)

```
So the real P0a diff is the uncommitted working-tree changes. There's no `test_doctor*.py`. Let
me look at the actual P0a diff (working tree vs HEAD) for main.py and loader.py, and read the
existing server-entrypoint test to understand what's already covered.
```

20. user (2026-06-21T14:42:54.007Z)

```
diff --git a/backend/main.py b/backend/main.py
index 6768d30..8e931f8 100644
--- a/backend/main.py
+++ b/backend/main.py
@@ -7,7 +7,16 @@ from urllib.parse import quote
 import uvicorn
 from dotenv import load_dotenv

-load_dotenv()
+# PACKAGING_PLAN.md P0a: prefer an app-home .env (the packaged launcher pins RALLY_APP_HOME),
+# else fall back to the historical CWD+walk-up search so the repo/dev path is unchanged.
+# stdlib-only import ? keeps this pre-pydantic line light.
+from backend.utils.app_home import default_env_path, resolve_output_dir
+
+_env_path = default_env_path()
+if _env_path.exists():
+    load_dotenv(_env_path)
+else:
+    load_dotenv()

 # Centralized logging setup for the indexer (hot paths use logger, CLI output uses print for
direct user-facing messages).
 logging.basicConfig(
@@ -67,13 +76,17 @@ app.add_middleware(
     allow_headers=["*"],
 )

-# Serves static frontend files directly (now under api/ per approved structure)
-os.makedirs("backend/api/static", exist_ok=True)
-app.mount("/static", StaticFiles(directory="backend/api/static"), name="static")
+# Serves static frontend files directly (now under api/ per approved structure).
+# PACKAGING_PLAN.md P0a: resolve PACKAGE-relative (not CWD-relative) so launching from an
+# arbitrary directory finds the shipped assets instead of creating a stray
./backend/api/static.
+# (The legacy in-engine dashboard; the real SPA gets bundled here in P1a.)
+_STATIC_DIR = os.path.join(os.path.dirname(os.path.abspath(__file__)), "api", "static")
+os.makedirs(_STATIC_DIR, exist_ok=True)
+app.mount("/static", StaticFiles(directory=_STATIC_DIR), name="static")
```

```
 @app.get("/", response_class=HTMLResponse)
 def serve_index():
-    index_path = "backend/api/static/index.html"
+    index_path = os.path.join(_STATIC_DIR, "index.html")
     if os.path.exists(index_path):
         with open(index_path, "r", encoding="utf-8") as f:
             return f.read()
@@ -618,7 +631,7 @@ def main():
             out_dir = os.path.dirname(out_path)
             out_name = os.path.basename(out_path)
         else:
-            out_dir = args.output_dir
+            out_dir = resolve_output_dir(args.output_dir)  # P0a: app-home aware (CWD-identical
when unset)
             out_path = os.path.join(out_dir, out_filename)
             out_name = out_filename

@@ -653,7 +666,9 @@ def main():
     # The CLI --output-dir overrides the configured output directory. It now lives
     # on the typed model (OutputConfig.output_dir), so every consumer ? including
     # /api/compile ? reads the same value instead of a write-only runtime hack.
-    global_config.output.output_dir = args.output_dir
+    # P0a: a RELATIVE output dir roots under the app-home (RALLY_APP_HOME); with the env
+    # unset, resolve_output_dir("output") == "output" (CWD-relative) ? byte-identical to
before.
+    global_config.output.output_dir = resolve_output_dir(args.output_dir)
     os.makedirs(global_config.output.output_dir, exist_ok=True)

     db_path = os.path.join(global_config.output.output_dir, global_config.output.db_name)
```

21. user (2026-06-21T14:42:54.745Z)

```
diff --git a/backend/config/loader.py b/backend/config/loader.py
index dd34507..f937f23 100644
--- a/backend/config/loader.py
+++ b/backend/config/loader.py
@@ -16,6 +16,8 @@ from typing import Any

 from pydantic import BaseModel

+from backend.utils.app_home import default_config_path
+
 from .models import AppConfig
 from .presets import (
     COMPUTE_TARGET_FOR_PROFILE,
@@ -27,7 +29,10 @@ from .presets import (

 logger = logging.getLogger(__name__)

-DEFAULT_CONFIG_PATH = Path("config.json")
+# Back-compat alias (was a module constant; no external consumers). Prefer
+# ``default_config_path()`` ? it honours ``RALLY_APP_HOME`` (PACKAGING_PLAN.md P0a). With the
+# env unset this is ``Path("config.json")`` (CWD-relative), exactly the pre-P0a value.
+DEFAULT_CONFIG_PATH = default_config_path()


 def _deep_merge(base: dict[str, Any], override: dict[str, Any]) -> dict[str, Any]:
@@ -109,7 +114,7 @@ def load_config(path: str | Path | None = None) -> AppConfig:
     Missing file or unreadable file ? returns a fully defaulted AppConfig.
     Unknown keys in the file are tolerated (extra="allow" on the model).
     """
-    cfg_path = Path(path) if path is not None else DEFAULT_CONFIG_PATH
+    cfg_path = Path(path) if path is not None else default_config_path()

     file_data: dict[str, Any] = {}
```

```
     if cfg_path.exists():
@@ -199,7 +204,7 @@ def save_default_config(path: str | Path | None = None) -> Path:
     Used by scripts/doctor.py diagnostics and the --setup CLI path.
     Overwrites if the file exists.
     """
-    cfg_path = Path(path) if path is not None else DEFAULT_CONFIG_PATH
+    cfg_path = Path(path) if path is not None else default_config_path()
     cfg = AppConfig()
     cfg_path.parent.mkdir(parents=True, exist_ok=True)
     cfg_path.write_text(cfg.model_dump_json(indent=4), encoding="utf-8")
diff --git a/scripts/doctor.py b/scripts/doctor.py
index b14bda8..309431b 100644
--- a/scripts/doctor.py
+++ b/scripts/doctor.py
@@ -49,12 +49,16 @@ def init_default_config():
     """
     try:
         from backend.config import save_default_config
+        from backend.utils.app_home import default_config_path
     except Exception as e:
         print(f"[ERROR] Could not import backend.config loader: {e}")
         print("[INFO] Falling back to no-op for config initialization.")
         return

-    config_path = os.path.join(REPO_ROOT, "config.json")
+    # PACKAGING_PLAN.md P0a: write to the SAME app-home config path the server READS
+    # (RALLY_APP_HOME, else CWD/config.json) ? previously this wrote to the repo root while
+    # the server read from the CWD, so --setup and the server disagreed once CWD ? repo root.
+    config_path = str(default_config_path())
     if os.path.exists(config_path):
         print(f"[INFO] {config_path} already exists. Skipping initialization.")
         return
```

22. assistant (2026-06-21T14:42:58.230Z)

Now I have the complete P0a diff. Let me read the existing `test_server_entrypoint.py` to see what server-start behaviour is already tested, and check the `DEFAULT_CONFIG_PATH` alias for a subtle import-time bug. Let me also check the PACKAGING_PLAN for the documented behaviour-change intent.

23. user (2026-06-21T14:42:59.277Z)

```
1       """Regression guard for the `python -m backend.main` double-module job-hang.
2
3       `python -m backend.main` runs main.py as `__main__`, so `main()` configures `__main__`'s
globals
4       (`db_instance`/`global_config`). But `job_worker`/`uploads` resolve shared state via
5       `import backend.main` ? a SEPARATE module object whose `db_instance` stays `None`. The
async-job
6       drain then crashes silently on `None.claim_due_job()` and every submitted job hangs in
`queued`
7       forever. The in-process tests (TestClient + monkeypatched `db_instance` + an inline
executor)
8       cannot see this ? only the real subprocess does. This spawns the DOCUMENTED invocation
and asserts
9       a submitted job actually LEAVES `queued`.
10      """
11      import json
12      import os
13      import socket
14      import subprocess
15      import sys
16      import time
17      import urllib.error
18      import urllib.request
```

```
19
20      import pytest
21
22      REPO_ROOT = os.path.dirname(os.path.dirname(os.path.abspath(__file__)))
23
24
25      def _free_port() -> int:
26          s = socket.socket()
27          s.bind(("127.0.0.1", 0))
28          port = int(s.getsockname()[1])
29          s.close()
30          return port
31
32
33      def _get(url: str, timeout: float = 5):
34          with urllib.request.urlopen(url, timeout=timeout) as r:
35              return json.loads(r.read().decode())
36
37
38      def _post(url: str, payload: dict, timeout: float = 15):
39          req = urllib.request.Request(
40              url, data=json.dumps(payload).encode(), headers={"content-type":
"application/json"})
41          with urllib.request.urlopen(req, timeout=timeout) as r:
42              return json.loads(r.read().decode())
43
44
45      def test_python_m_backend_main_drains_submitted_jobs(tmp_path):
46          """Spawning the engine the documented way (`python -m backend.main --server`) must
actually
47          DRAIN a submitted job ? it must leave `queued`, never hang. (A dummy file fails
fast; the
48          verdict is irrelevant ? what matters is that the worker claimed and ran it.)"""
49          port = _free_port()
50          out_dir = tmp_path / "out"
51          video = tmp_path / "dummy.mp4"
52          video.write_bytes(b"not-a-real-video")  # exists -> the drain claims it; processing
fails fast
53          log = open(tmp_path / "server.log", "wb")
54
55          proc = subprocess.Popen(
56              [sys.executable, "-m", "backend.main", "--server", "--port", str(port),
57               "--output-dir", str(out_dir)],
58              cwd=REPO_ROOT, stdout=log, stderr=subprocess.STDOUT,
59          )
60          base = f"http://127.0.0.1:{port}"
61          try:
62              deadline = time.time() + 90  # cold import of torch/cv2 can be slow on CI
63              while time.time() < deadline:
64                  if proc.poll() is not None:
65                      pytest.fail(f"server exited early (code {proc.returncode}); see
{tmp_path/'server.log'}")
66                  try:
67                      if _get(f"{base}/api/health").get("status") == "ok":
68                          break
69                  except (urllib.error.URLError, ConnectionError, OSError):
70                      time.sleep(0.5)
71              else:
72                  pytest.fail("server never became healthy")
73
74              job_id = _post(f"{base}/api/process",
75                             {"video_path": str(video), "segmenter": "motion"})["job_id"]
76
77              status = "queued"
78              deadline = time.time() + 45
```

```
79              while time.time() < deadline:
80                  status = _get(f"{base}/api/jobs/{job_id}")["status"]
81                  if status != "queued":
82                      break
83                  time.sleep(1)
84
85              assert status != "queued", (
86                  "job stayed 'queued' ? the async drain never ran under `python -m
backend.main` "
87                  "(the __main__/backend.main double-module entrypoint bug regressed)."
88              )
89          finally:
90              proc.terminate()
91              try:
92                  proc.wait(timeout=10)
93              except subprocess.TimeoutExpired:
94                  proc.kill()
95              log.close()
96
```

24. user (2026-06-21T14:42:59.862Z)

```
docs\PACKAGING_PLAN.md
```

25. user (2026-06-21T14:43:00.343Z)

```
docs\PACKAGING_PLAN.md:89:| **P0a** | **App-data HOME** (`RALLY_APP_HOME` env + OS default
`%APPDATA%/Khelsutra` \| `~/.local/share/khelsutra`) threaded through
`loader.DEFAULT_CONFIG_PATH`, `load_dotenv(path=)` (`main.py:10`), `OutputConfig.output_dir`/db
path, the static mount; **unify `doctor.py` REPO_ROOT write with the server read** | ? | No | ?
| ? |
backend\config\loader.py:30:DEFAULT_CONFIG_PATH = Path("config.json")
backend\config\loader.py:112:    cfg_path = Path(path) if path is not None else
DEFAULT_CONFIG_PATH
backend\config\loader.py:202:    cfg_path = Path(path) if path is not None else
DEFAULT_CONFIG_PATH
docs\CODE_MAP.md:132:  - `::DEFAULT_CONFIG_PATH = Path("config.json")` (? CWD-relative, NOT
repo-root). `::_get_builtin_defaults` = `AppConfig().model_dump(...)`.
```

26. assistant (2026-06-21T14:43:06.837Z)

I found a real bug. The Grep against the repo (the master HEAD, not the worktree) shows `loader.py:112` and `loader.py:202` still reference `DEFAULT_CONFIG_PATH` ? but wait, that grep ran against the default cwd (the main repo at `C:\Users\avidu\Projects\badminton-highlight-indexer`), not the worktree. Let me confirm the worktree's loader.py uses `default_config_path()` at the call sites and analyze the `DEFAULT_CONFIG_PATH` module-level alias for an import-time freeze bug.

The worktree loader.py (which I read in full) at lines 117 and 207 correctly calls `default_config_path()`. But line 35 has `DEFAULT_CONFIG_PATH = default_config_path()` evaluated **at import time**. Let me verify this is a real concern and check the PACKAGING_PLAN doc section and whether the doctor subdir behaviour change is documented.

27. user (2026-06-21T14:43:08.260Z)

```
File does not exist. Note: your current working directory is C:\Users\avidu\Projects\badminton-
highlight-indexer.
```

28. user (2026-06-21T14:43:09.059Z)

```
C:\Users\avidu\Projects\bhi-wt-p0a\backend\config\loader.py:35:DEFAULT_CONFIG_PATH =
default_config_path()
C:\Users\avidu\Projects\bhi-wt-p0a\docs\CODE_MAP.md:132:  - `::DEFAULT_CONFIG_PATH =
Path("config.json")` (? CWD-relative, NOT repo-root). `::_get_builtin_defaults` =
```

`AppConfig().model_dump(...)`.

29. assistant (2026-06-21T14:43:28.442Z)

The worktree's `DEFAULT_CONFIG_PATH` is now only the back-compat alias (call sites use `default_config_path()` directly), so the import-time freeze does NOT affect runtime config resolution. Good ? that's not a live bug, just a stale alias. Let me read the PACKAGING_PLAN P0a section in the worktree to verify whether the doctor subdir behaviour change is documented as intended.

30. user (2026-06-21T14:43:35.846Z)

File does not exist. Note: your current working directory is C:\Users\avidu\Projects\badminton-highlight-indexer.

31. assistant (2026-06-21T14:43:46.714Z)

The Read tool is resolving against the wrong cwd despite an absolute path ? likely the worktree path isn't in the allowed dirs for Read. Let me use Bash to read the relevant PACKAGING_PLAN section.

32. user (2026-06-21T14:43:47.468Z)

Exit code 2
sed: can't read docs/PACKAGING_PLAN.md: No such file or directory

33. assistant (2026-06-21T14:43:51.004Z)

The PACKAGING_PLAN.md is in the main repo, not the worktree (it's a doc that exists at master HEAD). The Grep earlier found it at `docs\PACKAGING_PLAN.md` relative to the main repo cwd. Let me check both locations and find where doctor's behaviour change is (or isn't) documented.

34. user (2026-06-21T14:43:52.305Z)

ls: cannot access '/c/Users/avidu/Projects/bhi-wt-p0a/docs/PACKAGING_PLAN.md': No such file or directory
---MAIN---
-rw-r--r-- 1 avidu 197609 26007 Jun 21 19:52 /c/Users/avidu/Projects/badminton-highlight-indexer/docs/PACKAGING_PLAN.md

35. assistant (2026-06-21T14:43:56.769Z)

Interesting ? `PACKAGING_PLAN.md` exists in the main repo but NOT in the worktree. That means the worktree branch (`feat/packaging-p0a-app-home`) is based on an older master where the doc may not exist, OR the plan doc lives only on the main checkout's branch. The worktree is "behind origin/master by 1 commit." Let me read the PACKAGING_PLAN P0a row from the main repo to understand the documented intent, then check whether SETUP_NEW_MACHINE.md or any doc in the worktree documents the doctor write-location change.

36. user (2026-06-21T14:43:58.331Z)

Legend: ? Todo · ? In progress · ? Done · ? Gated. **One small PR per row**, branched from `origin/master`, no stacking, default-OFF.

ID	Deliverable	Depends on	Gated?	Status	PR
P0a	**App-data HOME** (`RALLY_APP_HOME` env + OS default `%APPDATA%/Khelsutra` | `~/.local/share/khelsutra`) threaded through `loader.DEFAULT_CONFIG_PATH`, `load_dotenv(path=)` (`main.py:10`), `OutputConfig.output_dir`/db path, the static mount; **unify `doctor.py` REPO_ROOT write with the server read**	?	No	?	?
P0b	**Redact `GET /api/config`** + test asserting no credential-shaped field serializes + Host-header/localhost-origin guard	?	No	?	?
P0c	**Config/profile-driven bind + CORS** (default `127.0.0.1`; `0.0.0.0` only under edge-

auth profile) ? replace hardcoded `main.py:756` | ? | No | ? | ? |
P0d	**Declare `scipy`** in `[project.dependencies]` (serving use `[verified features/rally_seq.py:17]`, undeclared) + **license-scan CI lane** (pip-licenses/reuse) + **built-artifact smoke lane** (build ? `indexer --help` ? boot ? `/api/health`)	?	No	?	?
P0e	**Single-instance guard** (PID/lockfile) so a 2nd launch can't open the same SQLite DB and have `fail_stale_jobs` sweep the live process's jobs	P0a	No	?	?
P1a	**Bundle the SPA single-origin**: build `khelsutra/web` at package time ? vendor `web/dist/` into the wheel (hatchling force-include ? `backend/ui/`) + SPA history-fallback; retire legacy static mount; record bundle hash	P0a	No	?	?
P1b	**Net-new endpoints** the wizard/console need: `GET /api/capabilities` (honest hw/segmenter/ffmpeg/weights probe), `GET /api/videos` (list), `GET /api/reels/{video_id}` + `/api/version` handshake + SPA?engine contract test	P1a	No	?	?

37. user (2026-06-21T14:43:59.453Z)

40:**Secret hygiene:** never put a secret in the repo, a PR, or `config.json`. Set them as env vars on the box
136:Drop a permissive smoke `config.json` in the repo root (the CLI reads `./config.json` from the cwd):
138:cd ~/rally && cat > config.json <<'JSON'
189:**Config** ? `config.json` `storage.s3`:
296:rm -f ~/rally/config.json ~/sample.mp4   # remove the throwaway smoke artifacts

38. assistant (2026-06-21T14:44:05.356Z)

The plan documents "unify `doctor.py` REPO_ROOT write with the server read" as the explicit intent ? so moving the write from REPO_ROOT to `default_config_path()` is intended. The behaviour change (doctor run from a repo subdir now writes to CWD/config.json, not repo-root/config.json) is a direct consequence. Let me verify the doctor's actual subdir behaviour and whether `--setup` invokes doctor in a subprocess (which would matter for whether RALLY_APP_HOME propagates and what CWD doctor sees).

Let me check the existing test coverage for what touches doctor / load_dotenv / static mount, and confirm the `import os` ordering in doctor still works (the `import platform` is inside functions).

39. user (2026-06-21T14:44:08.102Z)

tests/__pycache__/test_app_home.cpython-313-pytest-9.0.3.pyc
tests/test_app_home.py

40. user (2026-06-21T14:44:08.493Z)

backend/config/loader.py:19:from backend.utils.app_home import default_config_path
backend/main.py:13:from backend.utils.app_home import default_env_path, resolve_output_dir
backend/main.py:15:_env_path = default_env_path()
backend/main.py:634:   out_dir = resolve_output_dir(args.output_dir) # P0a: app-home aware (CWD-identical when unset)
backend/main.py:670:   # unset, resolve_output_dir("output") == "output" (CWD-relative) ? byte-identical to before.
backend/main.py:671:   global_config.output.output_dir = resolve_output_dir(args.output_dir)
scripts/doctor.py:52:   from backend.utils.app_home import default_config_path

41. assistant (2026-06-21T14:44:13.890Z)

This confirms the test gap clearly: `test_app_home.py` only tests the resolver functions in isolation and the loader integration. **None** of the production wiring sites are tested:
- `backend/main.py:15-19` load_dotenv app-home branch
- `backend/main.py:83-93` package-relative static mount + serve_index
- `backend/main.py:634, 671` resolve_output_dir at trim and server-start sites
- `scripts/doctor.py:61` init_default_config writing to default_config_path()

Let me verify a couple of subtle correctness points before finalizing: (1) whether

`default_env_path()` returning `Path(".env")` and `.exists()` being CWD-relative could change load_dotenv behaviour vs the bare `load_dotenv()` walk-up; (2) confirm the `--setup` subprocess inherits env so RALLY_APP_HOME propagates to doctor.

42. user (2026-06-21T14:44:16.732Z)

```
find_dotenv default usecwd: True
bare load_dotenv walks up via find_dotenv = True
```

43. assistant (2026-06-21T14:44:27.704Z)

This confirms an important subtlety for the "byte-identical when unset" contract: when `RALLY_APP_HOME` is unset, `default_env_path()` returns `Path(".env")`. If a `.env` exists in CWD, the new code calls `load_dotenv(Path(".env"))` instead of bare `load_dotenv()`. Bare `load_dotenv()` uses `find_dotenv()` which **walks up parent directories**; `load_dotenv(Path(".env"))` does not. So:

- If `.env` is in CWD: both load it (the new path short-circuits to the explicit path). Minor difference: bare load_dotenv would also walk up, but if CWD has it, behaviour matches.
- If `.env` is NOT in CWD but IS in a parent: old behaviour = walk up and find it; new behaviour = `_env_path.exists()` is False (CWD has no `.env`), so it falls to bare `load_dotenv()` which walks up. **Same.**
- If `.env` is in CWD AND a parent: old = CWD wins (find_dotenv stops at first); new = CWD wins. Same.

So the load_dotenv branch is actually byte-identical when unset ? good, the fallback `else: load_dotenv()` preserves walk-up. This is correct but **completely untested** ? and it's exactly the kind of subtlety that a regression could silently break (e.g. someone "simplifies" to always `load_dotenv(_env_path)`).

I now have enough to write up findings. Let me do a final check: confirm there's no existing test exercising `serve_index` / static mount or `main()`'s server-start output_dir path, and verify the doctor `init_default_config` is import-testable (no side effects on import).

44. user (2026-06-21T14:44:30.398Z)

```
=== count test files touching main.main() server path ===
tests/test_config.py
tests/test_generate_highlights.py
```

45. assistant (2026-06-21T14:44:34.496Z)

Confirmed: no test exercises `serve_index`, the static mount, `init_default_config`, or `main()`'s server-start `resolve_output_dir`/db-path wiring. `test_server_entrypoint.py` spawns a subprocess but always with `RALLY_APP_HOME` unset and from `REPO_ROOT`, so it never validates the app-home redirection.

I have everything I need. Let me compile the findings.

46. user (2026-06-21T14:45:35.685Z)

```
Structured output provided successfully
```

47. assistant (2026-06-21T14:45:46.439Z)

I've completed the review. Findings returned via StructuredOutput.

Summary of what I found under the test-gaps-and-doctor lens:

Test gaps (the resolver primitives are well-tested; the wiring sites are not):
- `main.py:15-19` load_dotenv app-home branch ? untested; contract holds only via the `else: load_dotenv()` walk-up fallback (verified `find_dotenv` walks up), trivially regressible.
- `main.py:83-93` package-relative static mount + `serve_index` ? untested; the one CWD-sensitive change independent of `RALLY_APP_HOME`.

- `main.py:634, 671-674` `resolve_output_dir` at the --trim and server-start/db-path sites ? only the pure function is tested; the sites that put the DB under `RALLY_APP_HOME` have no automated guard, and `test_server_entrypoint.py` never sets the env so it can't catch a regression.
- `scripts/doctor.py:61` `init_default_config` ? no `test_doctor*.py` exists at all.

Doctor behaviour-change question: moving the write from `REPO_ROOT` to `default_config_path()` is *intended* (PACKAGING_PLAN P0a explicitly says "unify doctor.py REPO_ROOT write with the server read") and is the correct fix. But the consequent run-from-subdir change (writes `./config.json` into the subdir instead of always repo-root) is undocumented, and the `doctor.py:6-9` docstring still falsely claims it "operates on the repo root ... regardless of the caller's cwd."

No blocking correctness bug in the change itself ? the env-unset byte-identical contract does hold ? but the safety net is thin. The `DEFAULT_CONFIG_PATH` alias is import-time-frozen (nit, not a live bug since call sites use the live helper).

Relevant files: `C:/Users/avidu/Projects/bhi-wt-p0a/backend/main.py`, `C:/Users/avidu/Projects/bhi-wt-p0a/backend/config/loader.py`, `C:/Users/avidu/Projects/bhi-wt-p0a/scripts/doctor.py`, `C:/Users/avidu/Projects/bhi-wt-p0a/tests/test_app_home.py`, `C:/Users/avidu/Projects/bhi-wt-p0a/tests/test_server_entrypoint.py`.